An exact integer pre-filter for the NumberFields@home discriminant test

The Passiflora polDisc pre-filter: what it computes, why it is correct, and how that was checked

3 November 2026

Abstract

NumberFields@home searches for number fields of degree 10 by testing a very large number of monic integer polynomials with an exact discriminant test, which dominates the running time of its CPU application, GetDecics. Passiflora places a pre-filter in front of that test. The pre-filter discards a polynomial when the Legendre symbol of its discriminant is -1 modulo one of 30 primes chosen for the work unit; such a polynomial provably cannot pass. A pre-filter of this kind is acceptable only if it never discards a polynomial that passes. This paper describes an implementation in integer arithmetic (signed Montgomery reduction on the CPU, unsigned Montgomery reduction on the GPU) and proves that the program as written makes exactly the decisions of the exact algorithms, independently of the floating-point environment. It states the contract between the filter and the application and reports the evidence: an exhaustive test of the compiled filter on 25.5 billion polynomials at tiny primes, an exact oracle with deliberate faults, a differential test of GetDecics' own exact test, the 31 test work units in every build, and the GPU kernel on the CPU and on a device. It also proves what GetDecics' exact test computes, under one condition on the data that the search regions are designed to meet: the test's first 64-bit steps do not overflow. The integer filter is also faster than the floating-point filter it replaces: in the application the CPU filter with the exact test behind it takes 4 to 58% less time, depending on the work unit and the instruction set, and the GPU kernel 9 to 12% less device time. Every result carries its status: proved, checked numerically, quoted, or assumed.

Contents

1 Introduction **2**

2 The rejection rule **3**

3 Two Euclidean chains for one Legendre symbol **3**

3.1 `chain_block`, the fast chain . . . 4

3.2 `chain_general`, the general chain . . . 5

3.3 The filter . . . 5

4 Exact integer arithmetic **6**

4.1 The reductions . . . 6

4.2 Ranges, weights and Euler's criterion . . . 7

4.3 The masked loop and the theorem . . . 7

4.4 The size limits, and the guard . . . 8

4.5 The start-up check . . . 9

5 The GPU filter **10**

6 What the exact test of GetDecics computes **11**

7 The contract with the application **13**

8 Evidence **14**

9 Performance **15**

9.1 Timings . . . 15

9.2 The prime split . . . 18

10 What it all rests on **22**

A Reproducing the checks **23**

B The stack-offset call on Windows **23**

1 Introduction

NumberFields@home searches for number fields of degree 10 by enumerating monic polynomials $f = x^{10} + a_9x^9 + \cdots + a_0$ with integer coefficients. The program that volunteers run is *GetDecics* [1]. Its final step is an exact *discriminant test*, `polDiscTest_cpu`, built on GMP:

Definition 1 (Passing)**.** Let S be a finite set of primes (read from the work unit; $S = \{2, 5\}$ in the current searches and in every test work unit). A monic $f \in \mathbb{Z}[x]$ *passes* if $D = \operatorname{disc}(f) \neq 0$ and $|D| = \left(\prod_{p\in S} p^{\alpha_p}\right)m^2$ for some integers $\alpha_p \geq 0$, $m \geq 1$.

Almost no polynomial passes, and the stock CPU application spends about 98% of its time in this test [reported, not re-checked]. *Passiflora* is a patch to GetDecics (source at commit `638ca4e`) with new source files for a pre-filter and for the GPU code. The enumeration and the exact test remain GetDecics' own code; Passiflora decides which polynomials reach the test, and its output must be exactly that of the stock program.

The idea. If f passes, then modulo a suitable prime q its discriminant is a square or zero (Proposition 4). So a Legendre symbol $\left(\frac{\operatorname{disc} f}{q}\right) = -1$ at one such prime proves that f fails, and the filter computes these symbols. The CPU filter uses the 30 largest suitable primes below 2^{13} ($8161, 8089, \ldots, 5881$ for $S = \{2, 5\}$). Each rejects about half of the polynomials that reach it; the survivors of all 30 go to the exact test, which decides them as before. The GPU kernel applies the same filter at the 24 smallest suitable primes in $(256, 8192)$ ($281, \ldots, 1489$ for $S = \{2, 5\}$, none of them the CPU's) before it hands its survivors to the CPU.

Why exactness needs care. The filter must never reject a polynomial that passes, or the project loses fields. An earlier version stored residues in floating point and reduced with the rounding trick $x - q \cdot ((x\,q^{-1} + M) - M)$, which is exact only if every operation is rounded to nearest in the declared precision. On the real application that assumption fails in practice [checked numerically]: built for x87 arithmetic (as a 32-bit x86 build is), the application lost valid polynomials in four of the seven smallest test work units (6, 12, 6 and 5), among them the only real result of one unit; a library that switches the rounding mode to "up" before the filter is initialised makes it reject valid polynomials. A filter whose correctness depends on the floating-point environment of a volunteer's machine cannot be proved correct for the program that ships. The filter described here uses integer arithmetic only.

Results. Section 2 proves the rejection rule. Section 3 gives the two Euclidean chains that compute the symbol and proves them exact over $\mathbb{F}_q$ (Theorems A, B, C). Section 4 proves that the integer program computes exactly those decisions, for every input (Theorem D), and derives the size limits of its primes. Section 5 does the same for the GPU kernel (Theorem E). Section 6 proves what GetDecics' exact test computes. Section 7 states the contract with the application, Section 8 the evidence, and Section 9 the performance. Section 10 lists what everything rests on and what is still open. Appendix A gives the commands that reproduce the checks, and Appendix B a remedy, not in the patch, for a speed effect that only the Windows programs show. Each result is marked [proved], [checked numerically] (by a complete or stated finite computation, or by tests), [standard result, quoted], [reported, not re-checked] or [named assumption].

One assumption about the data. Everything is proved for every input except one link: that GetDecics' exact test computes the definition of passing. It does so only while its first two iterations, which it runs in 64-bit integers without overflow checks, stay in range: hypothesis (H) of Section 6. The search regions keep these values below 2^{60}, and none of the 14.8 billion polynomials of the 31 test work

units overflows. For a polynomial outside, GetDecics can give a wrong verdict, and Passiflora's counters can then differ from the stock program's. Whoever widens a search region, or gives the program other polynomials, has to check (H) first; the source says so where the filter is called and above the 64-bit steps.

2 The rejection rule

Let $\mathbb{F}$ be a field and fix an algebraically closed field $\mathbb{K} \supseteq \mathbb{F}$. For nonzero $V, P \in \mathbb{F}[x]$ with $V = \operatorname{lc}(V)\prod_{i=1}^{v}(x-\beta_i)$ over $\mathbb{K}$, the *resultant* is $\operatorname{Res}(V,P) = \operatorname{lc}(V)^{\deg P}\prod_i P(\beta_i)$, and $\operatorname{Res}(V,0) = 0$ for $\deg V \geq 1$. It lies in $\mathbb{F}$ [standard result, quoted] [3, Chapter IV]. Directly from the definition [proved]:

(R1) $\operatorname{Res}(c,P) = c^{\deg P}$ and $\operatorname{Res}(V,c) = c^{\deg V}$ for a nonzero constant c;

(R2) $\operatorname{Res}(U,V) = (-1)^{\deg U \deg V}\operatorname{Res}(V,U)$, because $\prod_{i,j}(\alpha_j-\beta_i)$ and $\prod_{i,j}(\beta_i-\alpha_j)$ differ in $\deg U \deg V$ signs;

(R3) if $\deg V \geq 1$ and $R = P \bmod V$, then $\operatorname{Res}(V,P) = 0$ when $R = 0$, and $\operatorname{Res}(V,P) = \operatorname{lc}(V)^{\deg P - \deg R}\operatorname{Res}(V,R)$ otherwise, because $P(\beta_i) = R(\beta_i)$.

The *discriminant* of a monic f of degree n with roots $\rho_1,\dots,\rho_n$ is $\operatorname{disc} f = \prod_{i<j}(\rho_i-\rho_j)^2$.

Proposition 2. *[proved] Let $f \in \mathbb{F}[x]$ be monic of degree $n \geq 1$ with $n \cdot 1 \neq 0$ in $\mathbb{F}$. Then $\deg f' = n-1$, $\operatorname{lc}(f') = n$, and $\operatorname{disc} f = (-1)^{n(n-1)/2}\operatorname{Res}(f,f') = (-1)^{n(n-1)/2}\operatorname{Res}(f',f)$.*

Proof. $f'(\rho_i) = \prod_{j\neq i}(\rho_i-\rho_j)$ by the product rule, so $\operatorname{Res}(f,f') = \prod_i f'(\rho_i) = \prod_{i<j}(\rho_i-\rho_j)(\rho_j-\rho_i) = (-1)^{n(n-1)/2}\operatorname{disc} f$. By (R2), $\operatorname{Res}(f',f) = (-1)^{n(n-1)}\operatorname{Res}(f,f') = \operatorname{Res}(f,f')$. □

Proposition 3. *[proved] Let $f \in \mathbb{Z}[x]$ be monic of degree $n \geq 2$ and q a prime with $q \nmid n$. Then $\operatorname{disc} f \in \mathbb{Z}$, and its residue modulo q is the discriminant of $f \bmod q$.*

Proof. $\operatorname{Res}(f,f')$ is the determinant of the Sylvester matrix [standard result, quoted] [2, Section 3.3], a polynomial with integer coefficients in the coefficients of f and f'. As $q \nmid n$, $f' \bmod q$ still has degree $n-1$, so the Sylvester matrix over $\mathbb{F}_q$ is the reduction of the one over $\mathbb{Z}$, and the determinant commutes with reduction. Apply Proposition 2 on both sides. □

For an odd prime q write $\chi(a) = \left(\frac{a}{q}\right)$. *Euler's criterion* $a^{(q-1)/2} \equiv \chi(a) \pmod q$ [standard result, quoted] gives: χ is multiplicative; $\chi(c^2a) = \chi(a)$ for $c \not\equiv 0$; $\chi(c^e) = \chi(c)^{e \bmod 2}$ for every integer e (negative e meaning powers of c^{-1}), so only the parity of an exponent matters; and $\chi(-1) = (-1)^{(q-1)/2}$, so -1 is a square exactly when $q \equiv 1 \pmod 4$.

Proposition 4 (The rejection rule)**.** *[proved] Let f pass for S, and let q be an odd prime with $q \equiv 1 \pmod 4$ at which every $p \in S$ is a nonzero square. Then $\left(\frac{\operatorname{disc} f}{q}\right) \in \{0,1\}$. So if $\left(\frac{\operatorname{disc} f}{q}\right) = -1$ at such a prime, f does not pass.*

Proof. $D = \pm\left(\prod_p p^{\alpha_p}\right)m^2$, so $\chi(D) = \chi(\pm1)\prod_p \chi(p)^{\alpha_p}\chi(m)^2 = \chi(\pm1)\chi(m)^2 \in \{0,1\}$, as $\chi(-1) = 1$ and $\chi(p) = 1$. □

A prime q with these properties is called *usable* for S. The conditions on S enter only here; the chains below work for any odd prime $q \nmid n$.

3 Two Euclidean chains for one Legendre symbol

Throughout this section q is an odd prime, $f \in \mathbb{F}_q[x]$ is monic of degree $n \geq 2$ with $q \nmid n$, and arithmetic is exact in $\mathbb{F}_q$. The filter computes the symbol of $\operatorname{Res}(f',f)$ by Euclid's algorithm on f and f', without inverses: instead of dividing by a leading coefficient it multiplies by it.

Lemma 5 (Pseudo-division step)**.** *[proved] Let $U, V \in \mathbb{F}_q[x]$ be nonzero, $u = \deg U \geq v = \deg V$, $l_u = \operatorname{lc} U$, $l_v = \operatorname{lc} V$, and $W = l_v U - l_u x^{u-v} V$, so $\deg W < u$. Then $W(\beta) = l_v U(\beta)$ at every root β of V; if $W \neq 0$, then $\operatorname{Res}(V,W) = l_v^{\,e}\operatorname{Res}(V,U)$ with $e = \deg W - u + v$; and if $W = 0$ and $v \geq 1$, then $\operatorname{Res}(V,U) = 0$.*

Proof. $V(\beta) = 0$ gives the first claim. Then $\operatorname{Res}(V,W) = l_v^{\deg W} \prod_i W(\beta_i) = l_v^{\deg W+v} \prod_i U(\beta_i) = l_v^{\deg W+v-u} \operatorname{Res}(V,U)$. If $W = 0$, then $U(\beta_1) = 0$. □

By the remarks on χ, a step with e even does not change the symbol, and a step with e odd changes it by $\chi(l_v)$.

Definition 6. The *remainder sequence* of (f, f') is $r_0 = f$, $r_1 = f'$, $r_{s+1} = r_{s-1} \bmod r_s$ while $r_s \neq 0$ and $\deg r_s \geq 1$. It is *normal* if $\deg r_s = n - s$ for $s = 0, \ldots, n-1$ (then r_n is a constant, possibly 0), and *abnormal* otherwise.

3.1 `chain_block`, the fast chain

`chain_block` stores coefficients by increasing powers ($P[k]$ is the coefficient of x^k) and runs 64 polynomials in the same instructions. For $s = 1, \ldots, n-1$, with $m = n - s$, $A = P_{s-1}$, $B = P_s$ ($P_0 = f$, $P_1 = f'$): $\lambda_s = B[m]$, $\tau = A[m+1]$; round 1, $R = \lambda_s A - \tau x B$; round 2, $P_{s+1} = \lambda_s R - R[m]\, B$; and, if $m \geq 2$, the check $\texttt{ok} \leftarrow \texttt{ok} \wedge (P_{s+1}[m-1] \neq 0)$. The coefficients that are not computed are zero: the coefficient of x^{m+1} in R is $\lambda_s \tau - \tau \lambda_s$, and that of x^m in P_{s+1} is $\lambda_s R[m] - R[m]\lambda_s$. At the end $\texttt{base} = P_n[0]$, $\texttt{acc} = \texttt{base}^{(q-1)/2}$, and the polynomial is rejected when `ok` holds and $\texttt{acc} = -1$.

Lemma 7. *[proved] Let B have degree exactly $m \geq 1$, $\lambda = \operatorname{lc} B$, $\deg A \leq m+1$, τ the coefficient of x^{m+1} in A, $R = \lambda A - \tau x B$, $t = R[m]$ and $R_2 = \lambda R - tB$. Then $R_2 = \lambda^2 (A \bmod B)$.*

Proof. $\lambda^2 A = \lambda R + \lambda \tau x B = (\lambda \tau x + t) B + R_2$ with $\deg R_2 < m$; by uniqueness of division $R_2 = (\lambda^2 A) \bmod B = \lambda^2 (A \bmod B)$. □

Lemma 8. *[proved] Put $\sigma_0 = \sigma_1 = 1$ and $\sigma_{s+1} = \lambda_s^2 \sigma_{s-1}$. If $\lambda_1, \ldots, \lambda_S \neq 0$ ($1 \leq S \leq n-1$), then $\deg r_s = n - s$ for $1 \leq s \leq S$, $P_s = \sigma_s r_s$ for $0 \leq s \leq S+1$, and every σ_s is a nonzero square.*

Proof. Induction on s. If $P_{s-1} = \sigma_{s-1} r_{s-1}$ and $P_s = \sigma_s r_s$ with σ's nonzero and $\lambda_s \neq 0$, then $\deg P_s = n - s$, so $\deg r_s = n - s$, and Lemma 7 gives $P_{s+1} = \lambda_s^2 (\sigma_{s-1} r_{s-1} \bmod \sigma_s r_s) = \lambda_s^2 \sigma_{s-1} (r_{s-1} \bmod r_s) = \sigma_{s+1} r_{s+1}$. □

Proposition 9 (The check detects exactly abnormality)**.** *[proved] At the end of `chain_block`, `ok` is true if and only if the remainder sequence of (f, f') is normal; if the sequence first becomes abnormal at r_{s+1}, the check first fails at step s.*

Proof. $\lambda_1 = n \neq 0$, and the check of step s tests λ_{s+1}, so `ok` holds iff $\lambda_2, \ldots, \lambda_{n-1} \neq 0$; then Lemma 8 gives $\deg r_s = n - s$ for all $s \leq n-1$. Conversely, if the sequence is normal and $\lambda_s = 0$ for a smallest $s \geq 2$, Lemma 8 with $S = s-1$ gives $\lambda_s = \sigma_s \operatorname{lc}(r_s) \neq 0$, a contradiction. The same argument at the first abnormal index gives the last claim. □

Lemma 10. *[proved] If the remainder sequence is normal, $\operatorname{Res}(f', f) = \left(\prod_{s=1}^{n-1} \operatorname{lc} r_s\right)^2 r_n$.*

Proof. For $s \leq n-2$, (R3) and (R2) give $\operatorname{Res}(r_s, r_{s-1}) = \operatorname{lc}(r_s)^2 \operatorname{Res}(r_s, r_{s+1}) = \operatorname{lc}(r_s)^2 \operatorname{Res}(r_{s+1}, r_s)$, as $(n-s)(n-s-1)$ is even: a normal sequence produces no signs. For $s = n-1$, $\operatorname{Res}(r_{n-1}, r_{n-2}) = \operatorname{lc}(r_{n-1})^2 r_n$ by (R3) and (R1) (both sides are 0 if $r_n = 0$). Multiply. □

Theorem A (`chain_block`)**.** *[proved] If `ok` is true at the end, then $\texttt{base} = \sigma_n r_n$ and $\chi(\texttt{base}) = \chi\big((-1)^{n(n-1)/2}\big) \chi(\operatorname{disc} f)$. For $q \equiv 1 \pmod 4$ this is $\chi(\operatorname{disc} f)$, $\texttt{acc} = \chi(\operatorname{disc} f)$, and a rejection means $\chi(\operatorname{disc} f) = -1$.*

Proof. By Proposition 9 and Lemma 8, $\texttt{base} = P_n[0] = \sigma_n r_n$ with σ_n a nonzero square, so $\chi(\texttt{base}) = \chi(r_n)$. By Proposition 2 and Lemma 10, $\operatorname{disc} f = (-1)^{n(n-1)/2} \Pi^2 r_n$ with $\Pi \neq 0$. For $q \equiv 1 \pmod 4$, $\chi(-1) = 1$; Euler's criterion gives $\texttt{acc} = \chi(\texttt{base})$. □

For $n = 10$ the sign $(-1)^{45}$ is -1, so the condition $q \equiv 1 \pmod 4$ is needed: with $q \equiv 3 \pmod 4$, `chain_block` returns the wrong symbol for every polynomial with a normal sequence and $\operatorname{disc} f \not\equiv 0$.

3.2 `chain_general`, the general chain

`chain_general` stores polynomials *left-aligned*: entry i of P is the coefficient of $x^{\deg P - i}$, and the entries beyond the degree are zero. Then one elementary step $W = l_v U - l_u x^{u-v} V$ is the same formula in every lane, whatever the degrees: $W(i) = l_v U(i+1) - l_u V(i+1)$ for $0 \le i < u$ is W left-aligned with nominal degree $u-1$ (the coefficient of x^{u-j} in $x^{u-v}V$ is $V(j)$). One lane runs:

(a) $U = f$, $V = f'$, $\mathtt{mm} = 1$; in each iteration form W and shift out its leading zeros, $w = \deg W$;

(b) if $W = 0$, stop: the polynomial is kept;

(c) with $e = w - u + v$, if e is odd, $\mathtt{mm} \leftarrow \mathtt{mm} \cdot l_v$;

(d) if $w \ge v$, $U \leftarrow W$; otherwise $(U, V) \leftarrow (V, W)$ (a *swap*);

(e) if now $V = c_0$ is a constant: if $\deg U$ is odd, $\mathtt{mm} \leftarrow \mathtt{mm} \cdot c_0$; stop.

Then $\mathtt{acc} = \mathtt{mm}^{(q-1)/2}$, and the polynomial is rejected when the lane stopped at (e) and $\mathtt{acc} = -1$.

Proposition 11 (Invariant and termination)**.** *[proved] At the start of every iteration $U, V \neq 0$, $u \ge v \ge 1$, and there are $\varepsilon \in \{\pm 1\}$ and $\gamma \in \mathbb{F}_q^\times$ with*

$$\operatorname{Res}(f', f) = \varepsilon \cdot \mathit{mm} \cdot \gamma^2 \cdot \operatorname{Res}(V, U), \tag{I}$$

where $\varepsilon = (-1)^N$, N the sum of vw over the swaps so far. The lane stops within $2n-2$ iterations.

Proof. At the start $U = f$, $V = f'$ ($l_v = n \neq 0$), $\mathtt{mm} = 1$. In an iteration that goes on, $W \neq 0$ and Lemma 5 gives $\operatorname{Res}(V, U) = l_v^{-e} \operatorname{Res}(V, W)$; write $l_v^{-e} = \delta^2$ (e even) or $l_v \delta^2$ (e odd). Step (c) multiplies $\mathtt{mm}$ by l_v exactly in the second case, so (I) holds for (W, V) with $\gamma\delta$; after a swap (R2) adds the sign $(-1)^{vw}$. The new pair satisfies $u \ge v$, and the loop goes on only with $v \ge 1$. Each iteration that goes on lowers $u + v$ by at least one; it starts at $2n-1$ and is at least 2, so there are at most $2n-2$ iterations. □

Theorem B (`chain_general`)**.** *[proved] The lane stops within $2n-2$ iterations (18 for $n = 10$; the code allows 40). If it stops at (b), then $\operatorname{disc} f = 0$ and the polynomial is kept. If it stops at (e), then $\mathit{mm} \neq 0$ and $\chi(\mathit{mm}) = \chi\big((-1)^{n(n-1)/2}\varepsilon\big)\chi(\operatorname{disc} f)$. For $q \equiv 1 \pmod 4$ this is $\chi(\operatorname{disc} f)$, $\mathtt{acc} = \chi(\operatorname{disc} f)$, and a rejection means $\chi(\operatorname{disc} f) = -1$.*

Proof. At (b), (I) holds and $\operatorname{Res}(V, U) = 0$ by Lemma 5, so $\operatorname{Res}(f', f) = 0$. At (e), (I) holds for the new pair with $V = c_0 \neq 0$, and $\operatorname{Res}(V, U) = c_0^{\deg U}$ by (R1), whose odd part step (e) puts into $\mathtt{mm}$; so $\operatorname{Res}(f', f) = \varepsilon \cdot \mathtt{mm} \cdot \gamma'^2$ with $\mathtt{mm}$ a product of nonzero leading coefficients. Apply Proposition 2 and $\chi(\pm 1) = 1$ for $q \equiv 1 \pmod 4$. □

The swap signs are why $q \equiv 1 \pmod 4$ matters here too [checked numerically]: with primes $q \equiv 3 \pmod 4$, `chain_general` was wrong in about a quarter of the degree-4 cases and 70% of the degree-10 cases tested.

3.3 The filter

`pdf_init(numP, pSet)` chooses the primes: a prime is *chosen* if it is usable for S and lies in the proved range (the *guard*, Section 4.4): the 30 largest below 2^{13}, one stage each. `pdf_filter` then runs every surviving polynomial, stage by stage, through `chain_block` (pass 1), re-runs the lanes whose check failed through `chain_general` (pass 2), and drops a polynomial as soon as a chain rejects it. A stage at which more than half of the lanes left `chain_block` in the last call sends every lane to `chain_general` directly (`prefgen`), and every 64th call probes `chain_block` again. Survivors of all stages get flag 1, all others 0; a polynomial whose leading coefficient is not 1 always gets flag 1.

Theorem C (soundness and exactness)**.** *[proved] In exact arithmetic, for every input, block size and routing state: `pdf_filter` gives f the flag 0 if and only if f is monic and $\left(\frac{\operatorname{disc} f}{q_j}\right) = -1$ at some stage j, and then f is dropped at the first such stage. In particular it never rejects a polynomial that passes.*

Proof. A flag 0 needs a leading coefficient 1, and the chains read $a_0, \dots, a_9$ with $a_{10} = 1$, so they see f. At a stage it reaches, f takes one of three routes: `chain_general` in pass 1 (`prefgen`); `chain_block` with `ok` false, then `chain_general` in pass 2; or `chain_block` with `ok` true. The prime is usable and at least 11, so $q \nmid 10$. By Theorems A and B, on every route f is dropped exactly when $\chi_j(\operatorname{disc} f) = -1$ (on the second route a kept lane with $\operatorname{disc} f \equiv 0$ has symbol 0). By Proposition 3 the symbol is that of the integer discriminant, and by Proposition 4 a polynomial with symbol -1 does not pass. Induction over the stages gives the first such stage. □

The converse direction, that the filter rejects *every* polynomial with a symbol -1, is what makes a test with an exact oracle possible (Section 8).

4 Exact integer arithmetic

Representation. Elements of $\mathbb{F}_q$ are integers in a small range around 0. For $R = 2^k$ let $s_k(x)$ be the residue of x modulo R in $[-R/2, R/2)$. The chains use $R = 2^{16}$ in 16-bit lanes (`mont`, 16 lanes per AVX2 register, compiled to `vpmulhw`/`vpmullw`). Each 64-bit coefficient is split into three 21-bit pieces and loaded by one reduction with $R = 2^{32}$ and 64-bit products (`sredc`, in `ldcoef` and `ldres`). (Until v0.4.0 the filter also had six stages with primes below 2^{25}, in 32-bit lanes with `sredc`; Section 9.2 explains why they were dropped.)

What the language must do. The proofs use three rules: (S1) an operation whose exact result fits its kind gives that result; (S2) a conversion to a narrower kind reduces modulo 2^k; (S3) `shifta` is an arithmetic shift and `iand` with $2^k - 1$ keeps the low k bits. The Fortran standard leaves (S2) and the bit model to the processor; the GNU Fortran manual documents (S3) but not (S2), which GCC documents for C. Standard-conforming replacements exist, but they made `pdf_filter` 2 to 6 times slower in a micro-benchmark (105–126 ns per polynomial as written, 572–697 with `modulo` and exact division, 225–262 with offsets; AVX2, an earlier configuration with six small primes) [checked numerically], because the compiler then no longer uses the 16-bit multiply-high instructions. So the code relies on (S2) and (S3), and they are checked three ways: the compiled reductions against exact models, the whole filter on complete sets at tiny primes (Section 8), and at every start of the program (Proposition 22), the only one of the three that runs in every build.

4.1 The reductions

Lemma 12 (Montgomery reduction)**.** *[proved] Let $q > 1$ be odd, $qq' \equiv 1 \pmod R$, $x \in \mathbb{Z}$, $m = s_k(xq')$ and $u = (x - mq)/R$. Then u is an integer, $u \equiv xR^{-1} \pmod q$, and $|u| \le |x|/R + q/2$; in particular $|x| < qR/2$ implies $|u| < q$.*

Proof. $mq \equiv xq'q \equiv x \pmod R$; $uR \equiv x \pmod q$ and R is invertible mod q; $|m| \le R/2$. □

The only multiple of q in $(-q, q)$ is 0, and those in $(-2q, 2q)$ are 0 and $\pm q$: these are the zero tests of the code.

Lemma 13 (`mont`, $R = 2^{16}$)**.** *[proved] Let q be odd, $q \le 32767$, $q' = s_{16}(q^{-1} \bmod 2^{16})$, and a, b 16-bit values. `mont` computes the 32-bit product $x = ab$, `lo` $= s_{16}(x)$, $m = s_{16}(\texttt{lo} \cdot q')$, $h_1 = \lfloor x/2^{16} \rfloor$, $h_2 = \lfloor mq/2^{16} \rfloor$ and returns $h_1 - h_2$. No operation overflows, the result is the u of Lemma 12, and $|u| < q$ whenever $|a|, |b| < 2q$ and $q \le 2^{13}$.*

Proof. $|x| \le 2^{30}$ and $|\texttt{lo} \cdot q'| \le 2^{30}$ fit 32 bits (this needs q' signed); by (S2) $m = s_{16}(xq')$. By (S3) h_1, h_2 are the floors, both at most 2^{14} in absolute value. As $x \equiv mq \pmod{2^{16}}$, the low halves agree and $h_1 - h_2 = (x - mq)/2^{16} = u$, with $|u| \le 2^{14} + q/2 \le 32767$. Finally $(2q - 1)^2 < 2^{15}q$ holds exactly for $q \le 2^{13}$. □

Lemma 14 (`sredc`, $R = 2^{32}$)**.** *[proved] Let $q < 2^{31}$ be odd, $q' = s_{32}(q^{-1} \bmod 2^{32})$, and x a 64-bit value with $|x| < 2^{31}q$. `sredc` returns $u \equiv x2^{-32} \pmod q$ with $|u| < q$, and no operation overflows.*

Proof. $s_{32}(x)q'$ is at most 2^{62} in absolute value; $m = s_{32}(xq')$; $|x - mq| < 2^{32}q < 2^{63}$; $x - mq$ is divisible by 2^{32}, so the arithmetic shift divides exactly; $|u| < q$ by Lemma 12. □

Lemma 15 (Loading). *[proved] For every 64-bit c, the pieces $p_0 = \texttt{iand}(c, 2^{21}-1)$, $p_1 = \texttt{iand}(\texttt{shifta}(c,21), 2^{21}-1)$, $p_2 = \texttt{shifta}(c,42)$ satisfy $c = p_0 + 2^{21}p_1 + 2^{42}p_2$ with $0 \le p_0, p_1 < 2^{21}$, $-2^{21} \le p_2 < 2^{21}$. With $d_i = 2^{21i}2^{32} \bmod q$, $\texttt{sredc}(p_0d_0 + p_1d_1 + p_2d_2)$ is congruent to c modulo q and lies in $(-q, q)$.*

Proof. By (S3) the pieces are the digits of c in base 2^{21} with a signed top digit. The sum has absolute value below $3 \cdot 2^{21}q < 2^{31}q$ and is congruent to $2^{32}c$; apply Lemma 14. □

4.2 Ranges, weights and Euler's criterion

Proposition 16 (Ranges). *[proved] At a stage whose prime q was chosen (so $11 \le q \le 8385$): every input coefficient is in $(-q, q)$, every stored coefficient, leading coefficient and* `base` *in $(-2q, 2q)$,* `mm` *and* `acc` *in $(-q, q)$, every argument of* `mont` *in $(-2q, 2q)$, and every difference of two results of* `mont` *in $(-2q, 2q)$ without overflow. So the reduction of Lemma 13 applies everywhere, and a value is 0 modulo q exactly when it is 0 or $\pm q$.*

Proof. For $q \le 2^{13}$, by induction over the statements: inputs come from Lemma 15; the constants $1, \dots, 10$ are below $2q$ as $q \ge 11$; `mont` of arguments in $(-2q, 2q)$ lies in $(-q, q)$ (Lemma 13), so a difference of two results lies in $(-2q, 2q)$, below 2^{14}; all other statements copy, move, merge or zero values. For $8192 < q \le 8385$ the claims follow from Theorem 21, which bounds every argument by a number below $qR/2$. □

Lemma 17 (Weights). *[proved] Run a chain exactly over $\mathbb{F}_q$ on $f \bmod q$ and the code on the loaded coefficients. Both take the same branches, and each polynomial of the code is congruent to its exact counterpart times R^{-w} for an integer $w \ge 0$, the same for all its coefficients; the same holds for the leading coefficients and for* `mm`*. So every zero test, degree, swap and* `ok` *agrees with the exact run, and $\chi($*`base`*$)$ and $\chi($*`mm`*$)$ are the exact symbols.*

Proof. A reduced product of a and b is $\equiv abR^{-1}$, and a reduced difference of products $\equiv (ab - cd)R^{-1}$. In every formula of the chains the two terms carry the same weight (in `chain_block`, both terms of $\lambda A[k] - \tau B[k-1]$ have weight $w(A) + w(B)$, and both terms of $\lambda R[k] - tB[k]$ weight $w(R) + w(B)$; in `chain_general`, both terms of $l_vU(i+1) - l_uV(i+1)$ weight $w(U) + w(V)$), so each result has one weight. R^{-w} is invertible, so zero tests agree, and the branches depend only on zero tests. $R = 2^{16} = (2^8)^2$ is a square, so no symbol changes. (The loading's reduction with 2^{32} carries no weight: its constants d_i contain the factor 2^{32}, so it returns $c \bmod q$ itself.) □

Lemma 18 (Euler's criterion in the Montgomery domain). *[proved] The code starts with* `acc` $= R \bmod q$ *and, for each bit of $(q-1)/2$ from the lowest, multiplies* `acc` *by* `base` *(if the bit is 1) and squares* `base`*, all with reduced products. At the end* `acc` $\equiv R\chi(b)$*, where b is the initial* `base`*; so $\chi(b) = -1$ exactly when* `acc` *is* `negr` $= (-R) \bmod q$ *or* `negr` $- q$*, which is the code's test.*

Proof. With $\beta = bR^{-1}$, the invariants `base` $\equiv R\beta^{2^i}$, `acc` $\equiv R\beta^{E_i}$ hold, as the reduced product of $R\beta^s$ and $R\beta^t$ is $R\beta^{s+t}$. At the end `acc` $\equiv R\beta^{(q-1)/2} = R\chi(\beta) = R\chi(b)$. The residues $R, 0, -R$ are distinct, and `acc` lies in $(-q, q)$. □

4.3 The masked loop and the theorem

`chain_general` runs all 64 lanes in one loop. A lane that has stopped is masked out; when all active lanes have the same degrees the slots are renamed instead of copied ("lockstep"); otherwise the first $\max \deg + 1$ entries are copied under a mask.

Lemma 19 (The masked loop). *[proved] Assume the conclusions of Proposition 16 and Lemma 17. At the start of every iteration of the code's loop, every active lane holds the U and V of the one-lane algorithm on its own polynomial after the same number of iterations (left-aligned, zeros up to index 11, each with one weight) and its* `mm`*; every inactive lane has stopped with its final* `mm` *and its stop reason recorded. Every lane stops within 18 iterations, and the decision is that of the one-lane algorithm.*

Proof. Induction over the iterations. At the start every lane holds f and $f'R^{-1}$. In an iteration, with `imax` the largest degree among the active lanes: the remainder is computed for indices below `imax`, and for a lane of smaller degree the extra entries are reductions of 0, hence 0, and indices from `imax` on are set to 0; shifting out leading zeros moves only the lanes whose leading entry is $\equiv 0$, and ends with $w = \deg W$ or $W = 0$; the zero-remainder, exponent and constant steps act under the mask of active lanes. The lockstep branch runs only when every lane is active with the degrees of lane 1, so renaming slots performs the same exchange in every lane; in the masked branch all entries beyond `imax` are zero, so copying indices up to `imax` copies whole polynomials. No write reaches an inactive lane's `mm`, degrees, flags or U, V. By Proposition 11 every lane stops within $2n - 2 = 18$ iterations, so the code's limit of 40 is never reached. □

Theorem D (the integer program is exact)**.** *[proved] Assume (S1)–(S3), and let the primes be those* `pdf_init` *chooses. For every input buffer, block size and routing state, at every stage and in every lane, the code's* `ok` *and keep decisions are those of the exact algorithms over* $\mathbb{F}_q$ *on* $f \bmod q$*. So Theorem C holds for the program as written. The filter executes no floating-point operation; its result depends neither on the rounding mode nor on x87 or SSE code nor on* `-ffast-math`*.*

Proof. The lanes start with $f \bmod q$ (Lemma 15); every value is in range (Proposition 16); the branches and symbols are the exact ones (Lemma 17); the -1 test is exact (Lemma 18). In `chain_block` every statement is an elementwise array operation without lane-dependent control flow, so each lane runs the algorithm; in `chain_general` Lemma 19 gives the same. The module has no `real` variable. □

4.4 The size limits, and the guard

Where the limits come from. Every limit of the filter is a consequence of its word size. A Montgomery reduction with $R = 2^k$ returns a value of size at most $|x|/R + q/2$ (Lemma 12), so it keeps the residues in $(-q, q)$ only while its argument stays below $qR/2$. The arguments are products of residues, a small multiple of q^2, so the prime may only grow to a fixed fraction of R. For one reduction with $R = 2^{16}$ and operands in $(-2q, 2q)$ this gives $q \le 2^{13} = R/8$ (Lemma 13). Along a chain the same arithmetic gives a recursion for the bound: in the worst case one iteration of `chain_general` turns a bound b into about $2b^2/R + q$. This recursion has a fixed point, so the values stay bounded however long a chain runs, exactly when $q \le R/8 = 2^{13}$. The filter's primes, below 2^{13}, lie in this region. Above it the bound grows with every iteration, but a chain stops after at most 18, so a somewhat larger prime still fits. How much larger has no closed form: Theorem 21 evaluates the recursion exactly for every odd q of a finite range, a proof by exhaustion rather than a measurement. The loading's reduction with $R = 2^{32}$ sets no limit of its own: its argument is below $3 \cdot 2^{21}q$, far under $2^{31}q$, for every $q < 2^{31}$ (Lemma 15).

Put $\rho(X) = \lfloor (X + qR/2)/R \rfloor$, the bound of Lemma 12 for the result of a reduction of an argument $|x| \le X$, with $R = 2^{16}$.

Lemma 20 (Bound propagation)**.** *[proved] Fix a stage with an odd prime $q > 5$. Define: $L = \lfloor (2(2^{21} - 1)(q-1) + 2^{21}(q-1) + 2^{31}q)/2^{32} \rfloor$ for the loading; for* `chain_block`*, $A_k = L$, $A_{10} = 1$, $B_{k-1} = \rho(kA_k)$, and for $m = 9, \dots, 1$ with $\lambda = B_m$, $\tau = A_{m+1}$: $R_0 = \rho(\lambda A_0)$, $R_k = \rho(\lambda A_k) + \rho(\tau B_{k-1})$, $P_k = \rho(\lambda R_k) + \rho(R_m B_k)$, then $A := B$, $B := P$; Euler's loop with the arguments B_0^2, $(q-1)B_0$, $(q-1)^2$; for* `chain_general`*, $b = \max(L, 1)$, $v = \rho(10L)$, then 18 times $w = 2\rho(vb)$, the argument $(q-1)\max(b, v, w)$, and $b := v := \max(b, v, w)$; Euler's loop with $(q-1)^2$. Call each argument of ρ a* site*, and each sum of two values of ρ a* difference site*. If every site is below $qR/2$ and every difference site at most 32767, then every argument of a reduction is bounded by its site, every result lies in $(-q, q)$, every difference fits 16 bits, and Lemmas 17, 18 and Theorem D hold at this stage.*

Proof. Induction over the statements, as in Proposition 16, with the intervals replaced by these numbers: a reduction of an argument with $|x| \le X < qR/2$ returns $|u| \le \rho(X) < q$, and ρ is increasing. In `chain_general` every slot of every lane is bounded by the current b after each iteration, including lanes that have stopped; by induction each active lane follows the algorithm and is active for at most 18 iterations. □

Theorem 21 (The limits of the whole filter)**.** *[checked numerically] The conditions of Lemma 20 hold for every odd q with $5 < q \le 8385$, and fail for every odd q in $[8387, 32767]$, first at the 18th iteration of* `chain_general`*. So Theorem D holds for every choice of primes $q \equiv 1 \pmod 4$ with $10 < q \le 8385$.*

The computation evaluates the recursion in 128-bit integers for every odd $q \le 32767$, and is repeated independently for every odd $q \le 9001$.

The guard. `pdf_init` chooses a usable prime only inside $[11, 8385]$, and writes a line on standard error for any it skips. The filter's primes are below 2^{13}, inside the region where the bound has a fixed point; the guard ensures that no future change of the prime search can leave the proved region silently. The repository's `checks.py` compares the guard's constants with the limit it computes, so a change of the guard itself fails there until the limit is proved anew.

How sharp the limits are [checked numerically]. The limit is that of the worst case, in which every reduction errs by the largest amount in the unfavourable direction. Since the worst-case bound grows above $R/8$, the limit depends on how many iterations a chain can run: the recursion of `chain_general` gives 8883 for 9 iterations and 8245 for 36, against 8385 for the actual 18. A finer analysis of the same kind (exact maxima of single reductions, a bound per coefficient) can therefore gain only a few per cent. The individual reduction fails earlier than the whole filter: `mont` with operands in $(-2q, 2q)$ first returns a value outside $(-q, q)$ at $q = 8241$ (complete search); the chains never form those operands. Above the filter's limit, a directed search on a model of the chains, at every prime $\equiv 1 \pmod 4$ from 8389 to 19961, finds the first results outside $(-q, q)$ at 15901 and the first wrong verdicts at 18457; those found at primes `pdf_init` could choose make the real filter fail as well. So the proved limit is at least 0.53 of the true threshold for such failures. A limit much closer to the threshold would need a proof that the reductions of one run cannot all err in the unfavourable direction, a property of the Montgomery quotients along a data-dependent chain over about q^{10} inputs for which no method is known. The search gives only upper bounds: its first failures moved to smaller primes as it was given more effort. (Until v0.4.0 the filter also had stages with primes below 2^{25}, $R = 2^{32}$ and 64-bit products, proved exact up to 1,097,010,265; Section 9.2.)

4.5 The start-up check

Proposition 22 (Start-up check)**.** *[proved] At the end of* `pdf_init` *the function* `arith_ok` *runs the compiled arithmetic on known inputs:* `mont` *and* `sredc` *on* 64 *inputs per prime including the extremes, the relations* $qq' \equiv 1$*, the split of* 64 *coefficients including* -2^{63} *and* $2^{63} - 1$*, and the loads of the filter themselves on a block of* 64 *polynomials with these coefficients. The expected values are computed with* $+$*,* $-$*,* $\times$*,* `modulo` *and exact division only, which do not depend on (S2) or (S3). If any comparison fails,* `pdf_init` *writes one line on standard error and chooses no prime; then every flag is* 1 *and the exact test alone decides.*

Proof. With no prime chosen the stage loop does not run, and every flag is set to 1. The rest is what the comparisons test. □

What the check is for. It does not guard the run-time environment, as a check of a floating-point filter would have to: integer instructions have no rounding mode, so a given binary computes the same values on every processor of its architecture. What (S2) and (S3) depend on is the compiler that built the binary, and that is fixed when the program is built. The release builds use several: the MinGW-w64 cross compiler for Windows, gfortran for x86-64 and aarch64 Linux, Homebrew's gfortran on macOS (arm64), the Android cross compiler (aarch64) and FreeBSD's gfortran, while the filter's own tests (Section 8) ran on x86-64 with GCC 15.2 and 16.2 only. The release workflow runs the other builds on the upstream work units; beyond that, the start-up check is the one test of their compiled integer code, and for a cross-compiled build the target machine is the first place where that code can run at all. The check costs no measurable time and fails safe: the filter switches itself off and the exact test decides every polynomial. It is not a proof. It runs the compiled loads exactly as the filter does, so a fault in the split (such as the top piece shifted logically) fails it, but not the copies of the reductions that the compiler inlines into the chains; (S2) and (S3) therefore remain named assumptions (Section 10).

Instruction sets. The application contains an AVX2 build and a baseline (SSE2) build of the filter and uses the AVX2 build when the CPU reports AVX2 and FMA. The AVX2 build is compiled with `-march=x86-64 -mavx2 -mno-popcnt`, so it can contain nothing the run-time test does not check: its

114 distinct instructions are base x86-64 instructions and VEX-encoded AVX and AVX2 instructions; the baseline object uses SSE and SSE2 only [checked numerically].

5 The GPU filter

The GPU kernel runs the enumeration's innermost loops and, for every polynomial it counts, the one-lane `chain_general` at the primes of its table. A polynomial it rejects is never shown to the CPU, so the kernel's filter must be sound by itself; its survivors go through the CPU filter and the exact test. It uses unsigned 32-bit Montgomery arithmetic with $R = 2^{16}$ and residues in $[0, 2q)$ (zero modulo q exactly when 0 or q), which needs no multiply-high instruction and no comparison per reduction. The table row of q holds q, the loading weights $2^{16j} \bmod q$, $(q-1)/2$, $q^- = -q^{-1} \bmod 2^{32}$ and $r^- = (-R) \bmod q$.

Lemma 23 (`redc`). *[proved] Let $q < 8192$ be odd, $0 \le t < 8q^2$ and $m = tq^- \bmod 2^{16}$. Then $t + mq < 2^{30}$ is divisible by 2^{16}, and $r = (t + mq)/2^{16} \equiv t\,2^{-16} \pmod q$ with $0 \le r < 2q$.*

Proof. $mq \equiv -t \pmod{2^{16}}$; $t + mq < 8q^2 + 2^{16}q < 2^{30}$; $r < t/2^{16} + q < q(8q/2^{16} + 1) < 2q$. Unsigned arithmetic in OpenCL C and CUDA wraps modulo 2^{32} by definition, and none of these values wraps. □

Theorem E (the GPU filter is exact). *[proved] For a prime $q \equiv 1 \pmod 4$ with $11 \le q < 8192$, its table row, and any 64-bit coefficients $c_1, \dots, c_{10}$, the kernel rejects $f = x^{10} + c_1x^9 + \cdots + c_{10}$ at q exactly when $\left(\frac{\operatorname{disc} f}{q}\right) = -1$. The table's primes are usable for S, so a polynomial the kernel rejects does not pass.*

Proof. Loading. With the 16-bit limbs u_j of $x \bmod 2^{64}$, the integer sum $\sum_j u_j(2^{16j} \bmod q)$ is at most $(2^{16}-1)(1+3(q-1)) < 2^{31}$; subtracting $2^{64} \bmod q$ for $x < 0$ and reducing gives $x \bmod q$. *Ranges.* Every residue is in $[0, 2q)$, and every argument of `redc` is below $8q^2$: $(10-i)a_i < 10q$, $l_vU(i) + (2q-l_u)V(i) < (2q-1)^2 + 2q(2q-1) < 8q^2$, products of two residues below $4q^2$. As $2q - l_u \equiv -l_u$, each new coefficient is the exact $W(i)$ times R^{-1}. *The chain* is the one-lane algorithm of Section 3 without masks; the zero test `w[0]==0 || w[0]==q` is exact on $[0, 2q)$, entries beyond a degree are 0, and it stops within $18 < 40$ iterations. As in Lemma 17, every polynomial is its exact counterpart times a power of R^{-1}, so the branches are exact and χ(`mm`) is the exact symbol, which is $\left(\frac{\operatorname{disc} f}{q}\right)$ by Theorem B. *Euler's criterion* runs as in Lemma 18 with `acc` $\in [0, 2q)$, so $\chi = -1$ exactly when `acc` $\in \{r^-, r^- + q\}$. □

The table. The table takes the smallest usable primes in $(256, 8192)$, at most 24. Every set S of one to three primes below 60 (the application allows $|S| \le 3$) has at least 24 there [checked numerically], while a range below 1024 would leave fewer than the 8 that GPU mode requires for 592 of these 833 sets. The smallest primes come first because Euler's criterion takes one step per bit of $(q-1)/2$: the 24 largest below 8192 instead took 1.0 to 1.3% more device time [checked numerically].

One kind of prime, none of the CPU's. The kernel, like the CPU filter, uses only primes below 2^{13}. Its arithmetic is unsigned 32-bit with $R = 2^{16}$; a larger prime needs 64-bit products, and an integer kernel with $R = 2^{32}$ was 47% slower than the floating-point kernel (Section 9.1). Larger primes would also gain nothing: a stage rejects about half of what reaches it whatever the size of its prime, and on the CPU, where both kinds were measured, the cost is lowest with the small primes first and does not rise when large primes are replaced by small ones (Section 9.2). The kernel's survivors then pass through the CPU filter's 30 stages before the exact test, where a prime of the kernel's table would reject nothing new. The CPU takes the largest usable primes below 2^{13} and the kernel the smallest above 256; for the sets S of the project's searches, with at least 115 usable primes in $(256, 2^{13})$, the two never share one ($281, \dots, 1489$ against $5881, \dots, 8161$ for $S = \{2, 5\}$).

Host and device code must match. Theorem E needs the kernel to read the table the program builds. For OpenCL this holds by construction: the kernel source is part of the program. The CUDA kernel is a fat binary embedded at build time and the MUSA kernel a library loaded at run time, so either could be left over from an older kernel that reads the table differently and rejects passing polynomials. The table layout therefore carries a number (format 2), the kernel's entry point carries it in its name (`nf_enum_t2`), and the MUSA library reports it and checks that every table row has the integer layout. A CUDA program with an old fat binary stops at start-up; a MUSA program refuses an old library and

runs on the CPU; an old program with a new library stops [checked numerically]. After any change of the kernel the CUDA fat binary and the MUSA library must be rebuilt.

Run-time checks on the device. The kernel was tested on one card, an NVIDIA GTX 1650 (Section 8). Under OpenCL its source is compiled at start-up by the graphics driver of each machine, and the CUDA and MUSA kernels run on cards the tests did not use, so the code a device runs has in general not been tested. Unlike the CPU's start-up check (Proposition 22), which stands in for tests of the release builds, these checks remain necessary however many builds are tested. At start-up every back end runs a self-test kernel of the same source once: `redc`, the loading, the verdict at every table prime for 240 polynomials and the full filter, 7,920 values for $S = \{2, 5\}$, which the host computes without the kernel's code (exact arithmetic modulo q, the symbol from the resultant). On a difference, or with device code that has no self-test, the task runs on the CPU. In the window at the start of a task, where CPU loops run next to the GPU, the program compares polynomial counts; with `NF_GPUCHECK=1` the CPU also applies the kernel's filter, compiled for the CPU, to the whole task, and the GPU's survivors must be the same, in the same order, or the task stops (about a microsecond per polynomial).

6 What the exact test of GetDecics computes

In the application the filter's rejections take the place of verdicts of GetDecics' own test, `polDiscTest_cpu`, so it matters what that test computes. It follows Cohen's sub-resultant algorithm [2, Algorithm 3.3.7] on $A = f$ and $B = f'$: with $g = h = 1$, it repeats $\delta = \deg A - \deg B$, $R = \operatorname{prem}(A, B)$ (the pseudo-remainder $\operatorname{lc}(B)^{\delta+1} A \bmod B$), $A \leftarrow B$, $B \leftarrow R/(gh^\delta)$, $g \leftarrow \operatorname{lc} A$, $h \leftarrow h^{1-\delta} g^\delta$, until $\deg B \le 0$. The numbers grow to about a thousand bits, so the test uses GMP; to save time it does the first two iterations in 64-bit integers (`int64`) and only then switches to GMP. At the end it removes every factor of the primes of S from its result `polDisc` and asks whether what remains is a perfect square.

Lemma 24 (Sub-resultants)**.** *[standard result, quoted] In this algorithm every division $R/(gh^\delta)$ and $g^\delta/h^{\delta-1}$ is exact in $\mathbb{Z}[x]$ resp. $\mathbb{Z}$, every g, h is nonzero, and the final $s\,t\,h^{1-\deg A} \operatorname{lc}(B)^{\deg A}$ is $\operatorname{Res}(A_0, B_0)$, where t is the product of the contents' powers and s a sign [2, Section 3.3].*

Proposition 25 (What the exact test computes)**.** *[proved] Let f be monic of degree 10 with `int64` coefficients, and suppose (H): no `int64` value computed in the first two iterations of `polDiscTest_cpu` leaves the `int64` range. Then $|\operatorname{disc} f| = r^2 |\texttt{polDisc}|$ for a nonzero rational r, and `polDiscTest_cpu` sets the flag to 1 exactly when f passes.*

Proof. By (H) every `int64` expression has its mathematical value and the conversions to GMP are exact, so the computation is one over $\mathbb{Z}$. *Initialisation:* $\operatorname{lc}(f') = 10$, so $\operatorname{cont}(f')$ divides 10; the coefficients of the odd powers of f' are even, so $2 \mid \operatorname{cont}(f')$ exactly when the even-power coefficients are even, which the code tests, and it tests 5 likewise; it divides B by $\operatorname{cont}(f')$, and $\operatorname{cont}(f) = 1$. *Pseudo-remainders:* each elimination step multiplies by $\operatorname{lc} B$ and subtracts a multiple of B that removes the leading term, $\delta + 1$ times in all (counting steps whose leading term is already zero as multiplications by $\operatorname{lc} B$); the result has degree below $\deg B$ and differs from $\operatorname{lc}(B)^{\delta+1} A$ by a multiple of B, so it is $\operatorname{prem}(A, B)$ by uniqueness of division in $\mathbb{Q}[x]$. *Reduction:* the code's scale factors are gh^δ and its update of h is Cohen's; by the lemma the divisions are exact, so C's `/` and `mpz_divexact` return the exact quotients. *The end:* if the final constant b_0 is 0 the resultant is 0 and the flag is 0. Otherwise, with $d = \deg A$, $\operatorname{Res}(f, f') = s\,t\,h^{1-d} b_0^d$ with $t = \operatorname{cont}(f')^{10}$, and the code takes `polDisc` $= h$ for even d and b_0 for odd d; the quotient is $(b_0/h)^d$ resp. $(b_0/h)^{d-1}$, an even power. With Proposition 2, $|\operatorname{disc} f| = \operatorname{cont}(f')^{10} |h^{1-d} b_0^d| = r^2 |\texttt{polDisc}|$. *The square test:* the code removes every factor of each $p \in S$ from $|\texttt{polDisc}|$ and tests the rest with `mpz_perfect_square_p`; that holds exactly when $v_p(\texttt{polDisc})$ is even for all $p \notin S$, which the factor r^2 does not change. This is Definition 1. □

Hypothesis (H). The two `int64` iterations multiply coefficients of f by coefficients of f', and then by coefficients of the first remainder; the code does not check that these products fit into 63 bits and relies on the polynomials it is given. The polynomials of the enumeration are bounded by its search region, and the author of GetDecics states that polynomials generated for the fields under test do not overflow these steps [reported, not re-checked]. The source shows the same design: it sends one branch of the second

iteration, where "the result would exceed a long", to GMP. For a polynomial outside the region the overflow is undefined behaviour in C as GetDecics is built; Passiflora compiles the test with `-fwrapv`, so that the value wraps around modulo 2^{64} by definition (as it does in practice anyway). The GMP steps that follow then work on a wrong number, and the verdict can be wrong in either direction.

Measurement. The program `test_poldisc.cpp` links the compiled `polDiscTest_cpu` of GetDecics, compares its flag with the definition computed independently by PARI (the discriminant, the primes of S removed, the square test), and replays the `int64` steps in 128-bit integers to detect an overflow. It runs the real candidates of the application, the results of the test work units, and families built to test the proposition (random coefficients, polynomials built to pass, square factors), which the enumeration does not produce (Table 1).

Table 1: The compiled `polDiscTest_cpu` of GetDecics against the definition (PARI), and the Passiflora filter, on 10,735,681 polynomials ($S = \{2, 5\}$). "ovf": an `int64` step of GetDecics overflows; "differ": the flag of GetDecics is not the definition's.

		pass			differ		filter	
family	polynomials	GetDecics	definition	ovf	no ovf	ovf	rejects	rejects passing
random, $\lvert a_i\rvert \le 3$	3,000,000	458	458	0	0	0	2,930,945	0
random, $\lvert a_i\rvert \le 100$	2,000,000	0	0	0	0	0	1,999,948	0
random, $\lvert a_i\rvert \le 10^6$	1,000,000	0	0	999,961	0	0	1,000,000	0
built to pass, small	2,000,000	1,999,939	1,999,939	0	0	0	0	0
built to pass, large	1,000,000	155,735	1,000,000	844,277	0	844,265	0	0
square factor	1,000,000	0	0	0	0	0	0	0
real candidates	733,855	0	0	0	0	0	733,855	0
the 83 results, transformed	1,826	1,826	1,826	0	0	0	0	0
all	10,735,681	2,157,958	3,002,223	1,844,238	0	844,265	6,664,748	0

What it shows [checked numerically]. Wherever (H) holds, GetDecics agrees with the definition: on all 8,891,443 polynomials without an overflow, as the proposition says. Outside (H), with coefficients far outside the search region, the `int64` steps overflow and the answer is no longer the definition's. Coefficient size alone is not the measure: the real candidates have coefficients up to $2.1 \cdot 10^9$ and never overflow, because their coefficients grow with the degree as those of polynomials with roots of moderate size do, which is what the search region produces. What matters is the largest value the `int64` steps form.

The evidence on real data [checked numerically]. On the 146,771 candidates captured from seven real work units the largest `int64` value is $2^{59.45}$, a factor of about 12 below the limit 2^{63}; on the 83 results of the test work units it is $2^{48.5}$; on all 733,855 real candidates and shifts of Table 1 no step overflows. A copy of the application that replays the `int64` steps of every polynomial before the filter (the 128-bit replay of `test_poldisc.cpp`) found no overflow among the 14,846,153,955 polynomials of the 31 test work units, all of which gave their truth files. This supports the author's statement, but it is a measurement, not a proof from the search bounds: (H) remains a named assumption, with three and a half bits of measured headroom.

The mechanism, illustrated. The following polynomial is constructed; the enumeration does not produce it. One of the 83 results in the truth files is $f = x^{10} + 600x^7 + 1350x^6 - 800x^5 + 115000x^4 + 405000x^3 + 265625x^2 - 540000x + 185000$. Multiplying its roots by 3 gives

$$g = 3^{10} f(x/3) = x^{10} + 16200x^7 + 109350x^6 - 194400x^5 + 83835000x^4 + 885735000x^3 + 1742765625x^2 - 10628820000x + 10924065000,$$

which defines the same field, and $\operatorname{disc} g = 3^{90} \operatorname{disc} f$, so g passes as f does. Its largest `int64` value would be about 2^{67}, and GetDecics rejects it; with the roots multiplied by 2 the largest value is about $2^{56.5}$ and GetDecics accepts it.

Passiflora and (H). Passiflora keeps the exact test as GetDecics has it (its patch changes only the test's conversions for a 32-bit `long` and adds comments, and `-fwrapv` only defines what an overflow does), so that its output is the stock program's. The filter rejects only polynomials that provably do not pass, so it never needs (H) for its own soundness; (H) enters only where the filter's verdicts are compared with those of the exact test (contract (C1)). Outside (H) the filter can differ from the stock program only by rejecting a polynomial that the exact test wrongly accepts. That did not happen in the tests (last column of Table 1), but it can be constructed [checked numerically]. When $B_9 = 10/\operatorname{cont}(f')$ is even, the `int64` steps see the constant term a_0 only through $B_9^2 a_0$, and $4 \mid B_9^2$, so adding 2^{62} to a_0 leaves every wrapped value unchanged: $h = x^{10} + x^9 - 3x^8 - 2x^7 + x^6 - x^5 - x^3 + 3x^2 + 2x - 2$ passes, and GetDecics also passes $h + 2^{62}$, which the filter rejects. The polynomials written out would be the same (PARI's field-discriminant test rejects $h + 2^{62}$), but the counters would differ. Such constant terms are far outside any search region. A check of (H) on every polynomial would cost about half of the filter's time; a checked version of the two iterations with a fallback to GMP would cost the stock program about 1% per exact test and is the change to propose to GetDecics, so that both programs stay identical.

7 The contract with the application

(C1) Soundness. A flag is 0 only if the `polDiscTest_cpu` of GetDecics would reject the polynomial.

(C2) Safety. No integer overflow, out-of-bounds access, read of an undefined value, other undefined behaviour, or illegal instruction.

(C3) Integration. The flags reach the exact test as its `polyMask` argument expects, and the 31 test work units give their `.truth` files.

Proposition 26 (The interface)**.** *[proved] After* `pdf_init(numP, pSet)`*,* `pdf_filter(pol, n, flag)` *with $n \le 0$ reads and writes nothing; with $n \ge 1$ it reads only the n rows of 11 coefficients, writes every flag with 0 or 1, and writes 0 only for a polynomial whose leading coefficient is 1 and which does not pass for $S = \{$*`pSet`*$\}$. Before the first* `pdf_init`*, and after a failed start-up check, every flag is 1. The module state left by earlier calls changes the routing, never a flag.*

Proof. The loads read only rows whose indices are in the work lists, which hold $1, \dots, n$; every flag is written. A flag 0 means a rejection at a chosen prime, which by Theorem D and Proposition 4 means that f does not pass. With no prime chosen every flag is 1 (Proposition 22). Theorem D holds for every routing state, and the work arrays are reallocated when they are too small. □

Proposition 27 (The glue)**.** *[proved] The application calls the filter into its flag array and then* `polDiscTest_cpu` *with the same array as* `polyMask`*. Afterwards the flag is 0 where the filter wrote 0 and the exact test's own flag elsewhere; under (C1) that is the exact test's flag everywhere, the output of the stock program.*

Proof. `polDiscTest_cpu` reads the mask at index k before it writes the flag at index k, touches no other index in that iteration, and writes a flag on every path. Every polynomial of the enumeration is monic (the enumeration sets the leading coefficient to 1 once), and the filter and the exact test receive the same S. □

With Proposition 25, these give (C1) under (H).

Safety [proved]. Overflow: the range proofs; the call counter is kept modulo 64. Bounds: every index is at most n or the lane count, and the chains' indices stay in 0..11 (Lemma 19). Undefined values: padding lanes hold x^{10}; a flag that `chain_general` does not set is set before it is read. Illegal instructions: the AVX2 object runs only when the CPU reports AVX2 and FMA, and contains nothing beyond AVX2. Language-level undefined behaviour: none apart from the conversions and shifts of (S2), (S3), which the start-up check tests.

Further conditions. (A1) At most three primes in S [proved]: GetDecics keeps S in `int pSet[3]`, and so do the patch's copies; the program refuses a work unit with more primes right after it reads S, before any array is filled (tested with four and five primes; all 31 work units have $|S| = 2$). Named assumptions: (A2) one thread: the filter keeps module state, and the application calls it only from the thread that runs the enumeration; (A3) hypothesis (H) of Proposition 25.

8 Evidence

The theorems are about the program text and the mathematics; the compiler, the device and the code of GetDecics are trusted only as far as the following checks show. CPU runs are on Linux (WSL2, GCC and gfortran 15.2) unless stated.

The compiled filter on complete sets at tiny primes [checked numerically]. The filter was built from the shipped source with its prime search moved to one tiny prime $q \in \{13, 17, 29, 37, 41\}$ (two constants changed), and every polynomial of a set was sent through the AVX2 and the baseline build, each with the normal routing and with every lane forced through `chain_general`: four runs per polynomial. Every flag and every rejection stage was compared with an exact oracle (Euclid with inverses). The sets: all 13^9 depressed monic polynomials modulo 13 (every monic polynomial modulo 13 is a shift of one of them); at 17 all with $a_9 = a_8 = 0$, at 29, 37, 41 all with $a_9 = \cdots = a_6 = 0$ (which send every chain into `chain_general` and its lockstep branch); and at each prime 200,000 random polynomials with all their shifts and coefficients lifted by large multiples of q. Altogether 25,518,310,785 polynomials and 102,073,243,140 verdicts: none wrong. (For v0.4.0 the same sets also ran through its 32-bit stage: 204,146,486,280 verdicts, none wrong.)

The arithmetic alone [checked numerically]. The reductions, cut out of the source (not retyped) and compiled with the filter's flags and at `-O0`, against exact models: `mont` on every pair of operands for every odd prime below 2^{13} (333,937,622,331 pairs) and every odd q in $[8193, 8303]$; `sredc` on windows around its limits and on random inputs; the loading on 101,072,896 coefficients for 6,169 moduli; the bound propagation of Theorem 21. With GCC 15.2 (Linux) and 16.2 (MSYS2): no error.

An exact oracle, and deliberate faults [checked numerically]. A test that only feeds polynomials that must pass cannot see a filter that rejects too little. The oracle test computes the symbol at every filter prime independently, builds its inputs to reach every stage and path of the filter (controlling the symbols by the Chinese remainder theorem), adds metamorphic checks, and reads a test hook that reports the stage of every rejection. Seven deliberate faults of the integer arithmetic, each one changed line, against 1,000,000 polynomials (total failures, AVX2 build):

fault	oracle test	"must pass" test
none	0	0
zero test $\pm q$ in `chain_block`	398	0
zero test $\pm q$ in `chain_general`	26,171	370
Euler: $-R$ recognised only as one representative	78,350	0
top piece of a coefficient shifted logically (start-up check: filter off)	668,550 missed	0
wrong loading constants (start-up check: filter off)	668,550 missed	0
parity of `chain_general`	277,230	161,849
last stage skipped	9,061	0

The oracle test sees all seven. The floating-point faults (x87, rounding up, down, toward zero) give 0 on the integer filter, as Theorem D says, while on the earlier floating-point filter the oracle test saw 66 false and 296 missed rejections under x87 and 37,329 false under rounding up. Primes moved across the size limits behave as the proofs predict: no wrong verdict inside them; beyond them wrong verdicts appear, at some primes only for directed inputs. The oracle test reaches all 30 stages: its `target` stratum is rejected about equally often at each. The 146,771 candidates captured from seven real work units pass through the filter with the oracle's verdicts.

The exact test [checked numerically]. Table 1 and Section 6.

The GPU filter [checked numerically]. The kernel's filter, compiled for the CPU from the kernel source itself, against the exact oracle prime by prime: 6,000,000 polynomials at the 24 table primes and 3,000,000 at tiny primes and just below the limit 8192, 228,000,000 checks, no false and no missed rejection. On an NVIDIA GTX 1650 the Windows application gave the truth files of all 31 work units through OpenCL and through CUDA (the CUDA kernel compiled by the release workflow's `nvcc` 12.9), with the start-up self-test passing in each, and the Linux CUDA program of v0.4.0 all 31 under WSL; a full comparison of one unit (3,759 survivors) found no difference, and injected faults and a kernel without the self-test were caught. The MUSA path was run with a stand-in runtime.

The work units [checked numerically]. The application, built from GetDecics at `638ca4e` with the patch by the repository's build scripts (Windows, MSYS2, with BOINC's own libraries), gave the `.truth` files of all 31 test work units with the AVX2 build and with the baseline build. With `NF_VERIFY=1` (the exact test also on every polynomial, stopping at the first difference), the five smallest units ran whole in both builds without a difference. The release workflow builds the CPU programs for every system and runs them on the work units before a release; for v0.4.0's filter, the same chains with $24 + 6$ primes, they gave every truth file on Linux x86-64, Windows, Linux ARM64, macOS and FreeBSD, and the two units run under emulation on RISC-V, PowerPC, IBM Z and 32-bit ARM.

Other sets S [checked numerically]. The proofs hold for every S. The tests were also run for $\{2, 11\}$, $\{5, 11\}$, $\{2, 5, 11\}$, $\{2, 3, 5\}$ and $\{3, 7\}$ without a failure, on fewer polynomials than for $\{2, 5\}$.

9 Performance

The integer filter replaced the floating-point one for exactness, not for speed, so this section first asks whether the change costs speed. It does not: the integer filter is faster everywhere, by 4 to 20% on the work units where almost every polynomial takes `chain_block`, by 48 to 58% on one where every polynomial takes `chain_general`, and by 9 to 12% in device time on the GPU. Section 9.1 gives the timings in the application and shows where the CPU filter's time goes, and what of the gain belongs to the arithmetic; Section 9.2 shows why all 30 stages of the CPU filter now use primes below 2^{13}: the six stages with larger primes that v0.4.0 still had cost slightly more and gain nothing. The timings of the CPU filter were made with v0.4.0's $24 + 6$ primes; the filter with $30 + 0$ is not separated from it by any measurement (Section 9.2). All results are measurements [checked numerically] on one machine, a laptop with an Intel i5-12500H and an NVIDIA GTX 1650; the CPU timings were made on Linux (WSL2) unless stated.

9.1 Timings

CPU [checked numerically]. Table 2 gives the application's own timing split (`NF_PROFILE`) over the first 60,006,400 polynomials of three work units, seven runs per filter, the filters alternating, one run at a time on an otherwise idle machine. The programs differ only in the filter: the earlier floating-point filter; an integer filter with Barrett reduction (32-bit lanes for the small primes, 64-bit products for the large ones); and the Montgomery filter in three steps, each with one change more than the step before. On `sf5` and `sf3` almost every polynomial takes `chain_block`; on `sf4` every polynomial takes `chain_general` at every prime.

The resulting filter is faster than the floating-point one in every row of Table 2: by 14% and 4% with AVX2 on `sf5` and `sf3` (the 4% is less than the spread of the runs, 5–31% between the fastest and the slowest run of one filter), by 19–20% with the baseline build, and by 48–58% on `sf4`. Barrett reduction is the slowest throughout: AVX2 has no 64-bit vector multiplication, and a 32-bit lane holds half as many residues per register as the 16-bit lanes of the Montgomery stage. Timings of different series differ by up to about 10%, so only filters of one series are compared.

Where the time goes [checked numerically]. Table 3 divides the filter's time on `sf5` into its parts. The first Montgomery filter was 45 cycles per polynomial slower than the floating-point one, and not because of its arithmetic: the chains of the small stages, where almost all the work is, were already 20 cycles cheaper (131 against 150). It lost 42 cycles in loading, where it gathered three arrays of 21-bit pieces through the block's indices, a loop the compiler does not vectorise (114 cycles, against 61), after splitting

Table 2: CPU filter plus exact test (the application's polDisc time) in seconds for the first 60,006,400 polynomials, median of seven runs; in brackets the change against the floating-point filter. Montgomery: $6 + 24$ primes (small + large) with scalar loads; the same primes with vectorised loads; then $24 + 6$ primes with these loads, the filter of v0.4.0 (the filter described here, $30 + 0$, is within the scatter of it; Section 9.2). Work units `sf5_DS-14x11_Grp33043of705883`, `sf3_DS-16x270_Grp74of3932160`, `sf4_DS-13x271-25_Grp98of200000`.

				Montgomery		
				scalar loads	vectorised loads	
work unit	build	floating point	Barrett	$6 + 24$ primes	$6 + 24$ primes	$24 + 6$ primes
`sf5`	AVX2	5.84	12.03 (+106%)	6.65 (+14%)	5.49 (−6%)	5.03 (−14%)
	baseline	8.86	24.47 (+176%)	8.80 (−1%)	7.96 (−10%)	7.21 (−19%)
`sf3`	AVX2	5.44	11.87 (+118%)	6.52 (+20%)	5.65 (+4%)	5.23 (−4%)
	baseline	8.20	22.52 (+175%)	7.89 (−4%)	7.33 (−11%)	6.59 (−20%)
`sf4`	AVX2	13.37	16.09 (+20%)	8.55 (−36%)	7.64 (−43%)	6.99 (−48%)
	baseline	21.88	32.81 (+50%)	11.25 (−49%)	10.86 (−50%)	9.28 (−58%)

Table 3: Where the filter's time goes: time-stamp-counter ticks ("cycles") per polynomial in each part of `pdf_filter`, AVX2 build, `sf5`, the first 20,008,960 polynomials in the application, median of three runs for each part (so the parts add up only approximately). The counters sit around every load, every chain call, the split of the coefficients in advance and the whole filter, in instrumented copies of the filters. Two series, the column "vectorised loads, $6 + 24$" in both, which shows how far the series differ. Floating point: `327b04f`; scalar loads: `27d917f`; $24 + 6$: `4094af4`; the vectorised loads with $6 + 24$ primes were not committed (on 28 October an earlier copy of the same loads, which the start-up check did not yet run).

	28 October 2026			29 October 2026	
	floating point	Montgomery			
		scalar loads	vectorised loads	vectorised loads	vectorised loads
part	$6 + 24$	$6 + 24$	$6 + 24$	$6 + 24$	$24 + 6$
split of every coefficient in advance	53.6	43.6	0.4	0.3	0.3
small stages: loads	61.3	113.7	103.7	93.1	95.9
small stages: `chain_block`	130.6	122.4	121.0	116.4	119.7
small stages: `chain_general`	19.8	8.2	8.2	7.5	7.6
large stages (loads and chains)	16.0	32.7	32.7	30.7	0.4
rest of `pdf_filter`	37.5	43.2	42.6	39.7	38.5
whole filter	318.9	363.9	308.6	287.3	262.5

every coefficient of the call into these pieces in advance (44, against 54 for the two doubles of the floating-point filter); 17 in the large stages, which cost twice as much as in the floating-point filter (33 against 16: AVX2 has no 64-bit vector multiplication, so the large primes need 64-bit products of 32-bit lanes); and 6 in the rest of the filter. *The vectorised loads* copy each polynomial's ten coefficients into a block buffer and split and reduce them on contiguous lanes, so the split in advance is gone: 104 cycles for loading instead of $114 + 44$, and the whole filter 55 cycles faster, with the arithmetic and the compiled chains unchanged. *The prime split*: with 24 small primes almost no polynomial reaches the large stages (0.4 instead of 30.7 cycles), for 6.2 more in the small stages, and the filter is 8.6% faster. Section 9.2 shows that $30 + 0$, which replaces the six large stages by small ones, is as good again: on `sf5` the six small stages take 0.05% of the filter's time where the six large ones took 0.17 to 0.23%.

What the floating-point filter could share. Neither step changes the arithmetic, so on `sf5` and `sf3` part of the lead is not the integer arithmetic's. The floating-point filter has the same loading structure (a split in advance, 54 cycles, then a gather through the block's indices, 61), and its loads could be rebuilt the same way; but AVX2 has no vector conversion from 64-bit integers to doubles, so a split inside the loads would first have to cut each coefficient into pieces of at most 32 bits, as the integer filter does. This was not tried, and what it would save is not known. It would gain less from the prime split. Its small stage runs in single precision, and its error argument covers only primes below 2^{10}, of which $S = \{2, 5\}$ has 14 usable ones (more would need a new bound); and its large stages cost 16 cycles, half as much as the Montgomery ones, so a split could save at most these 16 cycles (5%), less what the extra small stages

cost. What belongs to the integer arithmetic is the cheaper chains of the small stages and, on `sf4`, of `chain_general`, where the first Montgomery filter, with scalar loads and $6 + 24$ primes, was already 36% (AVX2) and 49% (baseline) faster than the floating-point one; against it stood the dearer large stages, which the filter no longer has.

GPU [checked numerically]. Table 4 gives one series of the kernel in four versions: the floating-point kernel, and the integer kernel in three steps, each with one change more than the step before (the integer arithmetic with 14 primes below 2^{10}; the table of 24 primes below 2^{13}; the start-up self-test). The versions alternate, and every run gave its truth file. The integer kernel takes 9–12% less device time than the floating-point one; the wider table, chosen for the sets S it covers rather than for speed (Section 5), adds 0.3–0.9%; the self-test, which runs once at start-up, leaves the device time unchanged. A whole run takes 3–10% less on the three longer units; on `sf5`, where start-up and the window check take most of the run, the differences are inside the spread of the runs. An integer kernel with $R = 2^{32}$, canonical residues and `mul_hi` was also exact but 47% slower than the floating-point kernel.

Measurement conditions. The test machine is a laptop (Intel i5-12500H, GTX 1650) on mains power. CPU load lengthens the device time on this machine: with all 16 hardware threads busy the device time on `sf4` rose from 2651 to 3123 ms (+18%) and returned to 2666 ms afterwards, on `sf7` from 6975 to 7015 ms. The GPU's core clock stayed at its maximum, 1785 MHz, with no throttle reason; the GPU was idle for longer stretches between launches and its memory clock dropped to its idle value at times, a likely cause that was not established. The GPU times of Table 4 were therefore taken with the CPU otherwise idle.

Table 4: GPU filter on a GTX 1650 (CPU otherwise idle, GPU core clock at its maximum 1785 MHz whenever busy), one series, medians; in brackets the change against the floating-point kernel. "device": the device time of the filter kernel's launches (OpenCL, a build instrumented for this measurement, fixed launch size, window check off), 7 runs, spread 0.5–2.5%. "OpenCL", "CUDA": the whole run with the program's defaults, 5 runs, spread up to 13% on `sf5`, up to 8% on the others. Integer kernel: for $S = \{2, 5\}$ the 14 usable primes in $(256, 1024)$; the 24 smallest usable primes below 8192; then with the start-up self-test, the program described here. Work units as in Table 2 and `sf7_DS-12x12_Grp881552of1000000`.

			integer, Montgomery $R = 2^{16}$		
			14 primes below 2^{10}	24 primes below 2^{13}	
work unit	time	floating point	no self-test	no self-test	self-test
`sf5`	device (ms)	1191	1044 (−12.3%)	1054 (−11.5%)	1054 (−11.5%)
	OpenCL (s)	2.68	2.70 (+1%)	2.67 (0%)	2.62 (−2%)
	CUDA (s)	2.51	2.49 (−1%)	2.49 (−1%)	2.46 (−2%)
`sf3`	device (ms)	15029	13592 (−9.6%)	13627 (−9.3%)	13629 (−9.3%)
	OpenCL (s)	16.76	15.41 (−8%)	15.37 (−8%)	15.32 (−9%)
	CUDA (s)	18.04	16.69 (−7%)	16.74 (−7%)	16.77 (−7%)
`sf4`	device (ms)	3027	2662 (−12.1%)	2674 (−11.7%)	2692 (−11.1%)
	OpenCL (s)	7.97	7.55 (−5%)	7.56 (−5%)	7.61 (−5%)
	CUDA (s)	7.86	7.58 (−4%)	7.45 (−5%)	7.64 (−3%)
`sf7`	device (ms)	7902	6989 (−11.6%)	7013 (−11.2%)	7011 (−11.3%)
	OpenCL (s)	9.59	8.63 (−10%)	8.78 (−8%)	8.80 (−8%)
	CUDA (s)	9.52	8.94 (−6%)	8.89 (−7%)	8.99 (−6%)

A repeat run [checked numerically]. On 2 November 2026 the floating-point versions and the program described here were timed again, two versions only, on the same machine and by the same procedure; Tables 2 and 4 stand as measured. The two CPU programs differ only in the filter. The two GPU programs both contain the CPU filter described here and differ in the GPU code and the host part of the patch. Table 5 gives the results. The changes agree with the earlier series to within 0.2 percentage points for the device time and 3 for the CPU and the whole runs: the CPU filter is 6–21% faster on `sf5` and `sf3` and 50–56% on `sf4`, the kernel takes 9.1–11.5% less device time, and a whole GPU run takes 1–9% less. As before, the AVX2 rows of `sf5` and `sf3`, the whole runs on `sf5` and the CUDA run on `sf4` differ by less than the spread of their runs. Both CPU programs gave the truth file of a whole work unit in both builds; all 142 GPU runs gave their truth files on the GPU (the integer kernel runs there only after its start-up self-test has passed), and the GPU core clock stayed at 1785 MHz.

Table 5: The repeat run: the floating-point version against the program described here, medians; in brackets the change. CPU: filter plus exact test in seconds for the first 60,006,400 polynomials, 7 runs, spread up to 15%. GPU, GTX 1650: "device" as in Table 4, 7 runs, spread 0.3–1.6%; "OpenCL", "CUDA": the whole run, 5 runs, spread up to 9%.

work unit	build	floating point	integer
`sf5`	AVX2	5.94	5.23 (−12%)
	baseline	8.73	6.98 (−20%)
`sf3`	AVX2	5.27	4.94 (−6%)
	baseline	8.31	6.55 (−21%)
`sf4`	AVX2	13.84	6.89 (−50%)
	baseline	20.66	9.13 (−56%)

work unit	time	floating point	integer
`sf5`	device (ms)	1194	1057 (−11.5%)
	OpenCL (s)	2.98	2.83 (−5%)
	CUDA (s)	2.59	2.48 (−4%)
`sf3`	device (ms)	15049	13677 (−9.1%)
	OpenCL (s)	16.98	15.67 (−8%)
	CUDA (s)	18.05	16.75 (−7%)
`sf4`	device (ms)	3038	2701 (−11.1%)
	OpenCL (s)	8.16	7.75 (−5%)
	CUDA (s)	7.66	7.58 (−1%)
`sf7`	device (ms)	7940	7051 (−11.2%)
	OpenCL (s)	9.84	8.97 (−9%)
	CUDA (s)	9.43	8.96 (−5%)

9.2 The prime split

Why there were two kinds of prime. Until v0.4.0 the CPU filter could divide its stages between small primes (16-bit lanes) and large primes (32-bit lanes, 64-bit products) in any proportion. Every stage keeps about half of the polynomials it is given, whatever the size of its prime (below), so the cost of a stage is its price per polynomial times the number that reach it, and only the price differs: a small stage holds 16 residues in a 256-bit register and multiplies in 16 bits, a large one holds 8 and multiplies in 64. Lemma 28 below proves what this implies: small stages go first, and replacing a large prime by an unused small one never raises the expected cost, so in the model all small, $30 + 0$, is best. The large stages are a legacy of the floating-point filter: its single-precision small stage was argued exact only for primes below 2^{10}, of which there are 14 usable ones for $S = \{2, 5\}$, too few for 30 stages, so it took 6 small and 24 large. The integer small stage is exact up to 2^{13} (Section 4.4), where there are 115, so the reason for the large stages is gone. v0.4.0 used 24 small and six large, written $24 + 6$, chosen by an early timing of a few splits; this section shows that it costs at most 0.14% more than $30 + 0$, less than any measurement resolves, because almost all the time is spent in the first stages, which every polynomial, then half of them, then a quarter pass through. The filter therefore now uses $30 + 0$, which removes the large stages, their size limit and their tests. The section gives a model of the cost, proves which orders and splits the model favours, and measures it, stage by stage and in the application.

Which primes there are [checked numerically]. The batch status pages of NumberFields@home (read on 30 October 2026) list seven searches for decic fields, all with $S = \{2, 5\}$; earlier, smaller searches used $S = \{2, 11\}$ and $\{5, 11\}$. For these three sets the rule of `pdf_init` (Proposition 4) leaves 115, 117 and 115 usable primes in $(256, 2^{13})$, so the filter always gets its 24 small primes, against 14, 13 and 14 in $(256, 2^{10})$, where an earlier version took them. Within the range the size of a prime hardly matters: a polynomial whose discriminant is not $\pm 2^a 5^b$ times a square survives a stage with probability about $(q + 1)/(2q)$ (half of the nonzero residues are squares, and $\operatorname{disc} f \equiv 0$ keeps it too), 0.50006 at $q \approx 8000$ and 0.50008 at $q \approx 6000$.

A model of the cost. One call of `pdf_filter` runs the stages in turn on the polynomials still kept, packed into blocks of 64 after every stage, and stops when none is left. Let N_j be the number of polynomials that enter stage j ($N_1 = n$, the size of the call) and N_{T+1} the number kept after the last of the T stages; the application runs the exact test on these. The model gives a stage of type t (small or large prime) that is entered by $N > 0$ polynomials in b blocks (those of pass 1 and the few of pass 2) the cost

$$c_t(N) = \alpha_t + \beta_t\, b + \gamma_t\, N, \qquad c_t(0) = 0,$$

since a block's chain costs the same however many of its 64 lanes are live, the gather of the coefficients does not, and a stage that runs at all pays for its constants and its loop. The exact test costs c_x per kept

polynomial. The coefficients depend on the work unit (on `sf4` every block takes `chain_general`) and on the build.

Lemma 28 (Order of the stages)**.** *[proved] Let one call run stages with the primes $q_1, \dots, q_T$ in this order, with N_j as above.*

(a) N_j depends only on the set $\{q_1, \dots, q_{j-1}\}$, not on its order, and $N_1 \ge N_2 \ge \cdots \ge N_{T+1}$.

(b) Suppose a stage of type t entered by N polynomials costs $c_t(N)$, where $D = c_\ell - c_s$ is nonnegative and nondecreasing on $\{0, 1, \dots, n\}$, and the exact test costs c_x per kept polynomial. Suppose further that for every set A of primes and any two primes $p, p' \notin A$, the numbers of polynomials of the call that $A \cup \{p\}$ and $A \cup \{p'\}$ keep have the same distribution. Then putting a small stage in front of an adjacent large one does not raise the expected cost, and neither does replacing a large prime by an unused small one. So, of all orders of s small and ℓ large primes, small ones first costs least in expectation, and the expected cost does not rise when large primes are replaced by small ones.

Proof. (a) By Theorem C, which holds for the program by Theorem D, a polynomial f of the enumeration (which is monic) enters stage j exactly when $\left(\frac{\operatorname{disc} f}{q_i}\right) \ne -1$ for $i = 1, \dots, j-1$, a condition on the set of these primes alone; the set of polynomials meeting it can only shrink as primes are added.

(b) Let stages i and $i+1$ have the large prime p and the small prime p', and let A be the primes before them. Exchanging the two changes nothing before stage i, and by (a) the polynomials that enter stage $i+2$ and every later stage, and those kept at the end, are the same either way. Stage i is entered by N_i polynomials either way; stage $i+1$ by the number K that $A \cup \{p\}$ keeps before the exchange and by the number K' that $A \cup \{p'\}$ keeps after it. The cost before minus the cost after is

$$c_\ell(N_i) + c_s(K) - c_s(N_i) - c_\ell(K') = D(N_i) - D(K') + c_s(K) - c_s(K').$$

The last two terms have expectation 0, since K and K' have the same distribution, and $D(N_i) \ge D(K')$ since $N_i \ge K'$ by (a) and D is nondecreasing. Replacing the large prime p of a stage k by a small prime p'' that no stage uses changes the cost of stage k from $c_\ell(N_k)$ to $c_s(N_k)$, which is $D(N_k) \ge 0$ less; every later stage, and the exact test, is entered by the polynomials kept by a set $A \cup \{p''\}$ instead of $A \cup \{p\}$ (A the other primes before it), which by assumption has the same distribution, and the expected cost is the sum of the stages' expected costs. Exchanges of adjacent pairs bring every small stage in front of every large one. □

The first assumption of (b) is a property of the cost functions, the second of the polynomials. In the model in which each polynomial whose discriminant is not $\pm 2^a 5^b$ times a square survives a prime q independently with probability $(q+1)/(2q)$, the distributions for two primes differ only through this probability, by less than 10^{-4} for the primes in question. The measurements test both assumptions.

The measurements [checked numerically]. Copies of the filter with only the constants of the split changed (`MAXS`, `MAXQ`), instrumented with time-stamp counters around every stage, load and chain and with counters of the polynomials, calls and blocks that enter every stage, were linked into the application (BOINC replaced by stand-ins, as for the work units of Section 8). They ran the first 20,008,960 polynomials of five work units, `sf3`, `sf4` and `sf5` of Table 2 and two from the running searches, `sf6_DS-7x10_Grp16925of160000` and `sf7_DS-12x12_Grp164787of1000000`, with ten splits ($24+6$ as in v0.4.0, $30+0$ as now, $18+12$, $12+18$, $6+24$, and 12, 16, 20, 24 and 40 small primes alone), both builds, five runs each, the splits alternating: 500 runs. Every call had $n = 10{,}240$ polynomials, and the counts were the same in all runs and both builds. The processor has two kinds of cores and a run may get either, so the filter's time varies by up to 30.5% between the runs of one split; stage 1 does the same work in every split (prime 8161, all polynomials), so every time is divided by the same run's stage-1 time, after which the five runs of a split agree to within 1.7%. On every unit the filter's time is 2.09 to 2.18 times that of stage 1: the first stage alone takes almost half.

The quantities of the model: a small stage keeps 0.50012 to 0.50017 of the polynomials that enter it (model 0.50006), a large one 0.49934 to 0.50074 (standard error 0.00063, model 0.50000); a full block of a large stage costs 3.1 to 4.2 times one of a small stage; the exact test costs 6.7 to 8.0 μs per kept polynomial; and on two units the polynomials that survive many stages come in *clusters*, several in one call, which stay together from stage to stage: of the polynomials kept by all 40 stages of $40+0$, 484 lie in 50 calls on `sf5` and 19 in 8 on `sf4`; the other three units have none. With these numbers and the

measured share of lanes that leave `chain_block`, the model gives the blocks of every split and unit to within 0.1% (those of stages 13 and later, where a call holds only a few polynomials, to within 4.4%), and with $\alpha_t, \beta_t, \gamma_t$ fitted on the small stages of $24 + 6$ and the large stages of $6 + 24$ alone it predicts the filter time of all ten splits to within 1.0% in every unit and build (Table 6). Differences below about 0.5% are within the scatter of the runs. The time of the stages after the 24th, measured directly, resolves those: on `sf5`, where clusters reach them, the six large stages of $24 + 6$ take 0.167% (AVX2) and 0.229% (baseline) of the filter's time, the six small stages of $30 + 0$ there 0.048% and 0.050%; on the other units at most 0.029% and 0.009%.

Table 6: Splits of the CPU filter: filter and exact test (the exact test as polDisc time minus filter time, in time-stamp ticks of the same run), change against $24 + 6$ in percent, measured (median of five runs, each in units of its stage 1) / model. $s + \ell$: s small and ℓ large primes. Work units as in Table 2, and `sf6_DS-7x10_Grp16925of160000`, `sf7_DS-12x12_Grp164787of1000000`.

build	split	`sf3`	`sf4`	`sf5`	`sf6`	`sf7`
AVX2	$6 + 24$	+6.91 / +7.09	+6.81 / +6.73	+8.58 / +8.41	+7.49 / +7.17	+7.07 / +7.10
	$12 + 18$	+1.76 / +1.63	+1.39 / +1.52	+2.36 / +1.75	+1.96 / +1.66	+1.65 / +1.63
	$18 + 12$	+0.07 / +0.05	−0.10 / +0.06	+0.43 / +0.14	−0.12 / +0.05	−0.07 / +0.05
	$30 + 0$	−0.15 / −0.00	−0.28 / −0.01	+0.23 / −0.10	+0.46 / −0.00	+0.07 / −0.00
	$12 + 0$	+1.76 / +1.74	+0.22 / +0.61	+1.36 / +1.23	+1.58 / +1.40	+1.61 / +1.73
	$16 + 0$	+0.27 / +0.08	−0.12 / −0.04	−0.09 / −0.14	+0.33 / +0.06	−0.05 / +0.08
	$20 + 0$	+0.18 / +0.00	−0.09 / −0.03	+0.31 / −0.17	+0.23 / +0.00	−0.17 / +0.00
	$24 + 0$	+0.20 / −0.00	+0.06 / −0.02	+0.18 / −0.14	+0.63 / −0.00	−0.23 / −0.00
	$40 + 0$	+0.16 / −0.00	−0.15 / −0.00	+0.48 / −0.03	+0.42 / −0.00	−0.27 / −0.00
baseline	$6 + 24$	+9.73 / +9.87	+9.92 / +9.81	+11.50 / +11.50	+9.84 / +9.77	+9.64 / +9.83
	$12 + 18$	+2.43 / +2.25	+2.39 / +2.26	+3.18 / +2.43	+2.12 / +2.23	+2.31 / +2.24
	$18 + 12$	+0.18 / +0.06	+0.09 / +0.08	+0.10 / +0.20	−0.07 / +0.06	−0.05 / +0.06
	$30 + 0$	+0.41 / −0.00	+0.19 / −0.02	+0.08 / −0.14	+0.00 / −0.00	−0.23 / −0.00
	$12 + 0$	+1.29 / +1.19	+0.18 / +0.26	+0.68 / +0.55	+1.09 / +0.89	+1.02 / +0.99
	$16 + 0$	+0.18 / +0.04	+0.24 / −0.07	−0.11 / −0.23	+0.27 / +0.02	−0.07 / +0.03
	$20 + 0$	+0.04 / +0.00	+0.08 / −0.04	+0.03 / −0.21	−0.25 / +0.00	+0.03 / +0.00
	$24 + 0$	+0.08 / −0.00	−0.09 / −0.03	−0.08 / −0.18	−0.19 / −0.00	+0.03 / −0.00
	$40 + 0$	+0.01 / −0.00	+0.20 / −0.01	+0.25 / −0.06	−0.08 / −0.00	−0.21 / −0.00

The optimum [checked numerically]. *Order.* The survival is the same for small and large primes within the error of the measurement, which is the second assumption of Lemma 28(b). The fitted costs give $D = c_\ell - c_s \geq 14{,}911$ time-stamp ticks (3.11 GHz) for every $N \geq 1$; D is nondecreasing on `sf5`, and on the other units it falls within one block by at most 5,306 ticks, less than a ninth of the step $\beta_\ell - \beta_s$ it then makes at the next block. So small primes go first, except possibly for such a fall when two counts lie in the same block. *Splits of* 30 *stages.* Under the model $30+0$ is the cheapest in every unit and build; $24+6$ costs 0.00 to 0.14% more, $18+12$ 0.04 to 0.34%, $12+18$ 1.5 to 2.6% and $6+24$ 6.8 to 11.6%. *The number of stages.* Adding a last small stage changes the expected cost by $E\, c_s(N_{T+1}) - c_x\, E(N_{T+1} - N_{T+2})$. Deep in the chain a call holds a cluster (with probability π, of mean size m) and about $n2^{-T}$ other polynomials; the stage costs about $\pi c_s(m) + n2^{-T} c_s(1)$ and saves about $c_x n 2^{-T-1}$. Without clusters (`sf3`, `sf6`, `sf7`) each further stage pays a little as long as $c_s(1) < c_x/2$, which holds there (6,095 to 9,084 against 10,452 to 12,452 ticks), so there is no best number. With clusters the saving halves from stage to stage while $\pi c_s(m)$ stays, and the last stage worth adding is about

$$T^* = \log_2 \frac{n\,(c_x/2 - c_s(1))}{\pi\, c_s(m)},$$

17.9 and 15.1 on `sf5` (AVX2, baseline; the model's best T is 18 and 16). On `sf4`, where $c_s(1) > c_x/2$, the model's best is 16 and 14. In every unit and build $T = 30$ costs at most 0.10% more than the model's best T; the measured totals for 16 to 40 small stages agree within the scatter, while 12 stages cost 0.2 to 1.8% more.

In the application [checked numerically]. The splits were also timed in the Windows application itself, with the procedure of Table 2 changed as follows. Seven programs, which differ only in the line of constants of the split, had the filter compiled as in `build-windows-gpu.sh` and were linked as that

script's CPU application (the GPU part replaced by its stub): $6+24$, $12+18$, $18+12$, $24+6$ and $30+0$, and $24+0$ and $40+0$ for fewer and more stages. Every run was pinned to one thread of a performance core at high priority, since a run that lands on an efficiency core takes about 2.5 times as long. There were nine rounds, each with all seven programs on every unit and build, the order of the programs rotated from round to round: 378 runs on 2 November 2026. Before the series every program gave the truth file of a whole work unit in both builds. Table 7 gives the results. Apart from `sf4` with AVX2 (below), $6+24$ costs 7.8 to 11.1% more than $24+6$ and $12+18$ 1.6 to 3.1% more, as in Table 6 (6.8 to 11.5% and 1.4 to 3.2% on these units). The other four splits lie between -1.3 and $+1.6$% of $24+6$. The runs of one program spread by 4 to 10.5% and the middle half of the paired ratios of one split covers up to 4 percentage points, so the timing does not order these four. On `sf4` with the baseline build $30+0$, $24+0$ and $40+0$ all come out 1.3 to 1.6% slower, with both quartiles above zero, where Table 6, which times the stages themselves, puts them within 0.2% of $24+6$; the time of the whole application does not resolve a difference of this size. With AVX2 the runs on `sf4` spread by 15 to 31% even when pinned.

The alignment of the stack [checked numerically]. The chains keep their arrays and spilled registers on the stack and read and write them 32 bytes at a time. On 64-bit Windows GCC aligns the stack to 16 bytes only and cannot realign it, so the compiled chains use unaligned moves, and depending on where address randomisation puts the stack in a run, every other such access crosses a cache line or none does. (The Linux build aligns these frames to 32 bytes and uses aligned moves.) In one fixed memory layout, moving the filter's stack by 16 bytes changed its time on `sf4` with AVX2 from 7.87 to 9.11 s (medians of six runs each, +8 to +19% in every pair), on `sf5` by 2.5%, and with the baseline build, whose 16-byte vectors never cross a line, not at all. On `sf4`, where every block takes `chain_general`, this outweighs the split, and only $6+24$ is separated there. It affects the Windows programs with the AVX2 build; the CPU timings of Tables 2, 5 and 6 were made on Linux. Compiling the AVX2 filter for 16-byte vectors removes the dependence but costs 26% on `sf4` and 29% on `sf5`; giving the chains' arrays static storage, which the compiler aligns, removes it too but makes the filter 46% slower on `sf4`, in the baseline build as well.

Why Tables 2 and 7 differ on `sf4`. They do not contradict each other. Table 2, made on Linux, compares the integer filter with the floating-point one, and on `sf4` its lead of 48 to 58% comes from the cheaper arithmetic of `chain_general`, far more than the stack can take away. Table 7, made on Windows, compares splits of one and the same filter, which differ by a few per cent; on `sf4` with AVX2 the stack moves the time by up to about 16% from run to run and hides these differences. The filter is not slow there: the median of $24+6$, 7.76 s, is of the order of the 6.99 s of Table 2, against 13.37 s for the floating-point filter.

A remedy, measured but not in the patch. The filter can be called, on 64-bit Windows with GCC only, through a short assembly function that puts the stack pointer at a fixed offset modulo 32, 0 or 16 (Appendix B gives the code and explains it). Which offset is faster depends on the compiled frames, so the first 24 pairs of calls with at least 256 polynomials run one call at each offset, in alternating order, and the offset with the lower median time per polynomial is kept. A Windows program built this way chose 16 in every run with AVX2 (with the baseline build, where the offset does not matter, either) and gave the truth file of a work unit in both builds; the offset changes the speed only, never a result. Timed as above with address randomisation on, in pairs against the program without it, the filter plus exact test on `sf4` with AVX2 took 7.17 to 7.78 s instead of 7.11 to 9.10 s: 7.2% less on average and 4.6% less in the median of the eight paired ratios. On `sf5` and `sf3` with AVX2 and on `sf4` with the baseline build the change was -1.3, -0.3 and -0.6% (6, 4 and 2 pairs). Nothing would change on Linux, where the wrapper is not compiled. It is left out of the patch, which keeps calling the filter directly: it would add assembly code and a timing loop to the program for a gain of a few per cent on one kind of work unit.

Splits with the model's best number of stages (14 to 18 small primes) were not timed: the model puts them at most 0.25% below $24+6$, less than this timing resolves.

The conclusion. Under the model $30+0$ is the cheapest split of 30 stages in all ten cases, and v0.4.0's $24+6$ is within 0.14% of it and within 0.25% of the model's optimum over all numbers of stages (`sf5`, baseline, at 16 stages); no measurement separates the two: neither the stage times of Table 6 nor the times of the application in Table 7, which agree with the model on $6+24$ and $12+18$ and put $18+12$, $30+0$, $24+0$ and $40+0$ within 1.6% of $24+6$. $24+6$ was chosen by an earlier, shorter timing of a few splits. Since the large stages bring nothing measurable, the filter now uses $30+0$: one kind of prime, one reduction in the chains, one size limit, and none of the tests and proofs that only the large stages needed. Every split is bounded below by the first stages, which every polynomial or every other polynomial passes

Table 7: Splits of the CPU filter timed in the Windows application: filter plus exact test (the polDisc time) for the first 60,006,400 polynomials, nine rounds, every run pinned to one thread of a performance core. Row $24+6$: the median in seconds. Other rows: change against $24+6$ in percent, the median over the rounds of the ratio to the $24+6$ run of the same round; in brackets the change of the medians. The runs of one program spread by 4–10.5%, on `sf4` with AVX2 by 15–31% (the alignment of the stack, see the text). Work units as in Table 2.

	`sf5`		`sf3`		`sf4`	
split	AVX2	baseline	AVX2	baseline	AVX2	baseline
$6+24$	+8.1 (+7.3)	+11.1 (+12.1)	+7.8 (+7.4)	+9.3 (+10.2)	+8.3 (+18.5)	+10.0 (+9.8)
$12+18$	+2.5 (+3.0)	+3.1 (+4.1)	+2.8 (+1.5)	+2.8 (+3.9)	+4.5 (+10.4)	+1.6 (+2.7)
$18+12$	+1.2 (+1.3)	+1.1 (+1.0)	+0.9 (+0.2)	−0.2 (+0.2)	+4.1 (+5.6)	+0.7 (−0.5)
$24+6$	5.31 s	7.11 s	5.24 s	6.95 s	7.76 s	10.19 s
$30+0$	−0.4 (−0.4)	−0.9 (+0.7)	+0.5 (−0.1)	+0.6 (−0.1)	+14.1 (+15.9)	+1.6 (+1.8)
$24+0$	−1.1 (+0.0)	−0.2 (+0.4)	−1.3 (−0.7)	+0.5 (−0.1)	+2.0 (+3.5)	+1.3 (+1.3)
$40+0$	+0.8 (+0.4)	−0.3 (−0.3)	+0.4 (+0.9)	+0.3 (+0.1)	−2.1 (−1.8)	+1.4 (+1.3)

through: a faster filter needs a cheaper small stage, not another split.

10 What it all rests on

claim	status	rests on
(C1) soundness	[proved]	Theorems A–D, Proposition 25, the interface; assumes (H), compiler correctness, (S2), (S3)
(C2) safety	[proved]	range proofs, size limits, the masked-loop lemma; assumes (S2), (S3) (start-up check), the CPU's report of AVX2/FMA
(C3) integration	[proved] [checked numerically]	the glue proposition; 31 units in both builds and through OpenCL and CUDA; assumes (A2)
GPU filter	[proved]	Theorem E; assumes the device compiler and device correct (tested on the device at start-up), host and device code of one format (checked at start-up)
compiler correctness	[named assumption]	tiny-prime test, arithmetic checks with two GCC versions, oracle test, start-up check; docs/BUILDING.md names the checked compilers
(S2), (S3)	[named assumption]	as above, on x86-64 with GCC 15.2 and 16.2; in every build the start-up check switches the filter off if the compiled loads or reductions disagree with exact values
(H)	[named assumption]	the enumeration's search region (the GetDecics author's statement); no overflow on 733,855 real candidates and shifts, largest value $2^{59.45}$ against 2^{63}; none among the 14,846,153,955 polynomials of the 31 test work units
(A2)	[named assumption]	no other thread of the program calls the filter

Open items.

- Where the filter first fails above its size limit. The limit 8385 is, up to the slack of the 18 iterations, the best a step-by-step worst-case bound can give (Section 4.4); the first failure for primes $\equiv 1 \pmod 4$ lies in $(8385, 15901]$. Locating it needs an argument about correlated reductions for which no method is known. The program is not affected: its primes are below 2^{13}, and the guard keeps every chosen prime inside the limit.
- (H) is measured on real candidates (about 12 times headroom) and checked on every polynomial of the 31 test work units, not proved from the search bounds.
- The GPU filter was tested on the CPU and on one NVIDIA card; AMD, Intel and MUSA devices were not tested. The program checks the filter on every device at start-up.
- The searches use $S = \{2, 5\}$; other sets were tested on fewer polynomials.
- On 64-bit Windows the speed of the AVX2 filter depends on where the stack lies in a run, by up to about 16% on a work unit where every polynomial takes `chain_general` (Section 9.2). A remedy is measured but not in the patch (Appendix B). The program's output is not affected.

A Reproducing the checks

The checks are scripts of the repository, in `pdfilter_Test` unless stated; they build their programs with `build-linux.sh` (Linux or WSL, with gcc, gfortran and PARI). `DIR` is GetDecics' `GetDecicsSrc` directory with the patch applied (`build/prepare-src.sh`).

command	checks
`bash tiny_checks.sh` (`QUICK=1`: a few minutes)	the compiled filter on complete sets at tiny primes
`bash arith_checks.sh` (`QUICK=1`: about a minute)	the compiled reductions; the bound propagation of Theorem 21
`bash fault_checks.sh 100 test_pdfilter_oracle.cpp`	the oracle test with every deliberate fault and the moved primes
`UPSTREAM=DIR CAND=... HITS=... bash poldisc_checks.sh`	Table 1; family `wrap`: the constructed overflow
`TEST=test_gpu_filter.cpp bash build-linux.sh`, then `build/test_gpu_filter 6000000 11 8`	Theorem E at the kernel's primes
`gcc -O2 -o limit_probe limit_probe.c`; `./limit_probe Q SEED N climb STEPS x`	the directed search above the size limit
`python3 checks.py`	the mathematics (resultants, both chains, the size-limit computation) with independent exact models; the limits in the source against the computed ones; every citation of this paper in the repository
`build/test-suite-unix.sh PROGRAM DATADIR` (repository root)	the 31 test work units against their truth files

Versions measured. GetDecics at `638ca4e`. Floating-point filter: CPU `327b04f`, GPU `718bab1`. Montgomery CPU filter with $6 + 24$ primes and scalar loads: `27d917f`; with vectorised loads and $24 + 6$ primes: `4094af4`. The filter with $30 + 0$ primes compiles its loads and chains to the same instructions as the small stages of v0.4.0's filter (GCC 15.2, both builds) [checked numerically]. GPU kernel with 14 primes: `e758190`; with the wider table: `0f874dc`; with the start-up self-test: `2c351c1`.

Not in the repository. The captured candidates, the Linux build recipe of the application with BOINC replaced by stand-ins, the Windows harnesses of the GPU runs and of the Windows timings, the Barrett version, and the instrumented copies and scripts used for the timings and for the overflow census of the test work units.

B The stack-offset call on Windows

This is the remedy of Section 9.2 ("The alignment of the stack"), which is not in the patch. It changes how the program calls the filter on 64-bit Windows, nothing else.

Why it helps. The chains keep blocks of 32 bytes on the stack. A block whose address is a multiple of 32 lies inside one cache line of 64 bytes; a block at 16 modulo 32 crosses a line every other time, and a crossing access is slower. The compiler places the blocks at fixed distances from the stack pointer that the filter receives, so where they fall depends only on that stack pointer modulo 32. On Linux GCC rounds it down to a multiple of 32 at the start of each chain; on Windows it cannot, and the value is 0 or 16 depending on where the system put the stack in that run. The remedy fixes the value itself.

The three parts. Part 1 is a function of ten instructions, `nfPdfCallAt(pol, n, flag, filter, off)`, which calls `filter(pol, n, flag)` with the stack pointer at $-\mathit{off}$ modulo 32: it saves the old stack pointer, rounds the stack pointer down to a multiple of 32, subtracts the offset and the 32 bytes that Windows reserves for every callee, calls the filter, and restores the old stack pointer. The `.seh` lines describe this to Windows, so that debuggers and crash reports can walk through the function. Part 2 decides the offset: the first 24 pairs of calls with at least 256 polynomials run one call at each offset, in alternating order, and time them; after the last pair the offset with the lower median of the ratios of the times is kept for all later calls. Part 3 connects them: after the filter is set up, the program's pointer to the filter is pointed at Part 2. The filter gets the same arguments at either offset, so the results cannot change.

The code. For the patched `TgtMartinet.cpp`: Parts 1 and 2 after the definition of `nfNow`, Part 3 at the end of `nfPdfSetup`, each inside `#if defined(_WIN64) && defined(__GNUC__) && defined(__x86_64__)` ... `#endif`, and `#include <algorithm>` with the other includes. The program that was timed made the same calls and the same decision; it also read an environment variable that fixes the offset and timed itself for the analysis, both left out here.

```
// Part 1: call filter(pol, n, flag) with the stack pointer at -off modulo 32 (off = 0 or 16).
extern "C" void nfPdfCallAt(const long long*, int, signed char*,
                            void (*)(const long long*, int, signed char*), long long);
asm(".text\n"
    "  .p2align 4\n"
    "  .globl nfPdfCallAt\n"
    "  .def nfPdfCallAt; .scl 2; .type 32; .endef\n"
    "  .seh_proc nfPdfCallAt\n"
    "nfPdfCallAt:\n"
    "  pushq %rbp\n"              // save rbp, then keep the old stack pointer in it
    "  .seh_pushreg %rbp\n"
    "  movq %rsp, %rbp\n"
    "  .seh_setframe %rbp, 0\n"
    "  .seh_endprologue\n"
    "  movq 48(%rbp), %rax\n"     // the 5th argument, off
    "  andq $-32, %rsp\n"         // round the stack pointer down to a multiple of 32,
    "  subq %rax, %rsp\n"         // subtract off
    "  subq $32, %rsp\n"          // and the 32 bytes Windows reserves for the callee
    "  call *%r9\n"               // filter(pol, n, flag): rcx, rdx, r8 are passed on unchanged
    "  movq %rbp, %rsp\n"         // restore the stack pointer and rbp
    "  popq %rbp\n"
    "  ret\n"
    "  .seh_endproc\n");

// Part 2: the first PAIRS pairs of calls with n >= 256 time both offsets; then the faster is kept.
enum { PAIRS = 24 };
static void (*nfPdfImpl)(const long long*, int, signed char*);    // the filter itself
static long long nfPdfOff = 16;                                   // the offset in use
static int nfPdfPairs = 0;                                        // pairs done; PAIRS: decided

static void nfPdfAligned(const long long *pol, int n, signed char *flag)
{
  static int half = 0;
  static double t0[PAIRS], t16[PAIRS];              // time per polynomial at offset 0 and 16
  if(nfPdfPairs == PAIRS || n < 256) { nfPdfCallAt(pol, n, flag, nfPdfImpl, nfPdfOff);  return; }
  long long o = ((nfPdfPairs + half) % 2) ? 16 : 0;  // pair k: 0 then 16, or 16 then 0
  double t = nfNow();
  nfPdfCallAt(pol, n, flag, nfPdfImpl, o);
  t = (nfNow() - t) / n;
  if(o == 0) t0[nfPdfPairs] = t; else t16[nfPdfPairs] = t;
  if(++half < 2) return;
  half = 0;
  if(++nfPdfPairs < PAIRS) return;
  double r[PAIRS];                                  // decide by the median of t0/t16
  for(int k = 0; k < PAIRS; ++k)  r[k] = t0[k] / t16[k];
  std::sort(r, r + PAIRS);
  nfPdfOff = (0.5*(r[PAIRS/2 - 1] + r[PAIRS/2]) > 1) ? 16 : 0;
}

// Part 3: at the end of nfPdfSetup, after pdfInit(numPrimes, pSet):
  nfPdfImpl = nfPdfFilter;  nfPdfFilter = nfPdfAligned;
```

What was checked. The function's unwind entry (`objdump -x`) covers all of it, with frame register `rbp` and the push of `rbp`, and the filter is called only through it. Printing the chosen offset on `stderr` from inside a timed call added up to 0.3 s to the measured filter time in the Windows timing runs, so a program that reports the offset should print it after the call. The Linux program built from a patch with this code (inside the `#if`) was identical byte for byte to the one without it.

References

[1] GetDecics, the NumberFields@home application, https://github.com/drivere/get-decics-numberfields (commit `638ca4e`).

[2] H. Cohen, *A Course in Computational Algebraic Number Theory*, GTM 138, Springer, 1993.

[3] S. Lang, *Algebra*, revised third edition, GTM 211, Springer, 2002.